

Resumen de estudio para el parcial

Extracción, procesamiento y representación vectorial de texto

Unidad 1. Extracción y procesamiento de texto

Unidad 2. Representación vectorial de texto

Materia: Procesamiento del Lenguaje Natural

Material base: apuntes de teoría de la cátedra (docente Juan Pablo Manson)

Fecha: 5 de octubre de 2026

 **Clave:** concepto central, candidato a pregunta de parcial.

[Ejemplo propio] : explicación o ejemplo que no está en el material.

Índice

Parte I. Extracción de texto	3
1. Extracción de texto: qué es y de dónde sale	3
2. Archivos TXT y codificación de caracteres	4
3. Formatos con marcado: HTML y Markdown	4
4. Documentos PDF	5
5. Texto en imágenes: OCR	7
6. Documentos de Word (DOCX)	8
7. Conversores de documentos	8
8. Audio y video	9
9. Otras fuentes: Wikipedia, bases de datos, tablas y JSON	10
10. Web scraping	10
11. Parseo de texto	11
Parte II. Procesamiento de texto	13
12. Limpieza y normalización	13
13. Stopwords (palabras de parada)	13
14. Estandarización y corrección ortográfica	14
15. Tokenización	15
16. Stemming y lematización	16
17. Emojis y emoticones	16
18. Analítica de texto	17
19. Segmentación de texto (chunking)	20
Parte III. Representación vectorial de texto	22
20. Por qué convertir texto en vectores	22
21. One-hot encoding	22
22. Count Vectorizer (vectorización por conteo)	23
23. TF-IDF	23
24. Vectorización de hash	25
25. Comparación de las codificaciones	26
26. Word embeddings	27
27. Similitud de coseno	28
28. Tipos de modelos de embeddings	30
29. Word2Vec: Skip-gram y CBOW	30
30. FastText	32
31. Línea temporal y embeddings contextuales (BERT)	33
32. Visualización de embeddings	34
33. Embeddings de oraciones y Sentence-BERT	35
Parte IV. Herramientas de estudio	38
Glosario	38
Página de repaso	40
Preguntas de autoevaluación	41
Observaciones sobre el material	41

Parte I. Extracción de texto

Esta parte corresponde a la primera mitad de la Unidad 1. Responde una pregunta práctica: cómo saco el texto de donde está guardado (un archivo, una web, un audio) y lo convierto en un string de Python con el que pueda trabajar.

1. Extracción de texto: qué es y de dónde sale

IDEA CENTRAL

Extraer texto es recuperar el texto plano que está guardado dentro de otro formato. Es el primer paso de cualquier proyecto de PLN (procesamiento del lenguaje natural).

EXPLICACIÓN

[Ejemplo propio] Una factura en PDF es, para la computadora, un archivo con instrucciones de dibujo, fuentes e imágenes. Antes de analizar qué dice, hay que "despegar" las letras de ese envoltorio y quedarse con una cadena de caracteres.

DEFINICIÓN FORMAL

La *extracción de texto* (text extraction) es el proceso de recuperar y aislar texto plano a partir de contenedores o formatos de almacenamiento: PDFs, páginas web, archivos HTML, audio transcrito o documentos escaneados. La *extracción de información* (information extraction) es otra tarea: parte de un texto ya extraído y busca entidades, relaciones o eventos estructurados.



Clave

Extracción de texto = sacar el texto del contenedor. Extracción de información = sacar datos estructurados (entidades, relaciones, eventos) de un texto que ya extrajimos. La segunda siempre viene después de la primera.

FUENTES TÍPICAS SEGÚN EL MATERIAL

Fuente	Qué incluye	Cómo se accede
Documentos de texto	Word, PDF, .txt, RTF	Librerías por formato
Páginas web	Blogs, noticias, foros, Wikipedia	Web scraping
Redes sociales	Comentarios, publicaciones, mensajes	APIs con tokens o credenciales
Correo electrónico	Cuerpo, asuntos y encabezados	Lectura del mensaje
Bases de datos	Descripciones de productos, registros de atención al cliente	Consultas SQL
Transcripciones	Entrevistas, podcasts, videos de YouTube	Reconocimiento del habla o APIs de subtítulos
Literatura	Libros y revistas en PDF, epub, mobi	Digitalización y extracción

RELACIÓN CON LOS DEMÁS TEMAS

La técnica y la librería dependen de la fuente. Cada sección que sigue trata una fuente distinta. Todo lo que se extrae acá pasa después por la limpieza y el procesamiento de la Parte II.

2. Archivos TXT y codificación de caracteres

IDEA CENTRAL

Un archivo de texto guarda bytes. Para convertir esos bytes en letras hay que saber con qué codificación (encoding) se guardaron.

EXPLICACIÓN

[Ejemplo propio] El encoding es un diccionario de claves: el byte 243 es "ó" en uno y otra cosa en otro. Si se escribe con uno y se lee con otro, aparecen símbolos raros o un error.

El formato TXT es el más básico: solo texto, sin negrita, cursiva ni colores. En Python 3 se lee con `open()`, que por defecto asume UTF-8. Si el archivo usa otra codificación, se indica con el argumento `encoding`:

```
with open('archivo.txt', 'r', encoding='iso-8859-1') as archivo:
    contenido = archivo.read()
```

DEFINICIÓN FORMAL

La codificación define qué secuencia de bytes representa cada carácter. En UTF-8 algunos caracteres ocupan más de un byte. En ISO-8859-1 cada carácter ocupa uno. Si leemos un archivo con la codificación incorrecta, Python puede lanzar un `UnicodeDecodeError`.

EJEMPLO DEL MATERIAL, CON EL CÁLCULO

La cadena "¿Cómo estás?" tiene 12 caracteres. Los caracteres especiales ocupan distinto según el encoding:

Carácter	UTF-8 (hex)	Bytes UTF-8	ISO-8859-1 (hex)	Bytes ISO
¿	C2 BF	2	BF	1
ó	C3 B3	2	F3	1
á	C3 A1	2	E1	1
Los otros 9 (C, m, o, espacio, e, s, t, s, ?)	1 byte c/u	9	1 byte c/u	9
Total		15		12

Cuenta: en UTF-8 son $3 \times 2 + 9 \times 1 = 15$ bytes. En ISO-8859-1 son $12 \times 1 = 12$ bytes. El mismo texto tiene distinta longitud en bytes y se decodifica con distinta tabla.

Clave

Antes de leer un archivo hay que conocer su encoding. UTF-8 es el predeterminado de Python 3. Un encoding equivocado produce `UnicodeDecodeError` o caracteres corruptos.

RELACIÓN CON LOS DEMÁS TEMAS

El encoding vuelve a aparecer en la eliminación de acentos (Parte II): ahí veremos que un mismo carácter visible puede tener dos representaciones Unicode distintas.

3. Formatos con marcado: HTML y Markdown

IDEA CENTRAL

HTML y Markdown mezclan el texto con marcas de formato. Para extraer el texto hay que quitar esas marcas.

3.1. HTML

HTML (HyperText Markup Language, lenguaje de marcado de hipertexto) define párrafos, encabezados, enlaces o tablas con etiquetas (tags). Un *elemento* es etiqueta de apertura (con atributos, como `href="contact.html"`) + contenido + etiqueta de cierre. El texto suele estar en `<body>`, dentro de una estructura que varía con cada página.

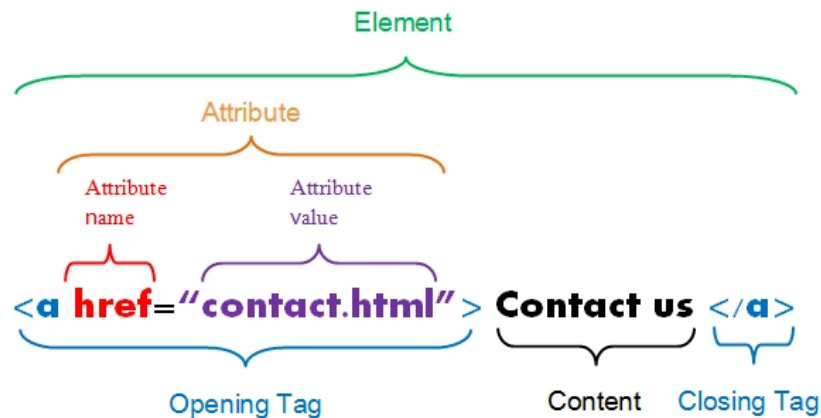


Figura 1. Anatomía de un elemento HTML: etiqueta de apertura con un atributo (nombre y valor), contenido y etiqueta de cierre. Fuente: material de la Unidad 1.

Etiquetas de formato del material: `<p>` (párrafo), `<h1>` a `<h6>` (encabezados de mayor a menor nivel), `<strong>` y `<em>` (negrita y cursiva), `<u>` y `<s>` (subrayado y tachado), `<pre>` (preformateado, conserva espacios y saltos) y `<blockquote>` (cita).

Para quitar las etiquetas se usa **BeautifulSoup** con el parser `lxml`. El atributo `.text` devuelve solo el texto:

```
soup = BeautifulSoup(contenido_html, 'lxml')
texto_plano = soup.text
```

Ojo: este método borra todas las etiquetas, también `<p>` y `<br>`, que marcan párrafos y saltos de línea. Si necesitamos conservar esa estructura, hay que tratar esos casos aparte.

3.2. Markdown

Markdown (.md) es texto con una sintaxis simple de formato (`# Título`, `**negrita**`). Es el formato predeterminado para documentar código en GitHub. Se lee como cualquier TXT. Para convertirlo:

- A HTML: librerías `markdown` o `mistune` (`md.markdown(texto)`).
- A texto plano: primero a HTML y después a texto con `html2text`. Con `h.ignore_links = True` se descartan los enlaces.

RELACIÓN CON LOS DEMÁS TEMAS

Markdown reaparece dos veces más: como formato de salida de los conversores de documentos y como formato con segmentación especializada (`MarkdownTextSplitter`).

4. Documentos PDF

IDEA CENTRAL

Un PDF guarda instrucciones para dibujar cada página, no párrafos. Por eso extraer su texto puede perder estructura, y si es un escaneo puede no haber texto para extraer.

EXPLICACIÓN

[Ejemplo propio] Un PDF se parece a un plano: dice "poné la letra A en tal coordenada". Al pedir el texto, la librería junta esas letras; con dos columnas o una tabla, el orden que arma puede no ser el de lectura.

DEFINICIÓN FORMAL: ESTRUCTURA DE UN PDF

El formato PDF (Portable Document Format) lo desarrolló Adobe Systems. Preserva fuentes, imágenes, gráficos y diseño sin importar la aplicación o la plataforma. Sus partes son:

1. **Cabecera:** versión del PDF, por ejemplo %PDF-1.7.
2. **Cuerpo:** los objetos del documento (texto, imágenes, gráficos), cada uno con un número único.
3. **Tabla de referencias cruzadas** (cross-reference table, xref): ubicación de cada objeto, para acceder rápido sin leer todo el archivo.
4. **Trailer:** ubicación de la tabla xref y del objeto raíz.
5. **EOF** (End of File, fin de archivo): la línea %%EOF.

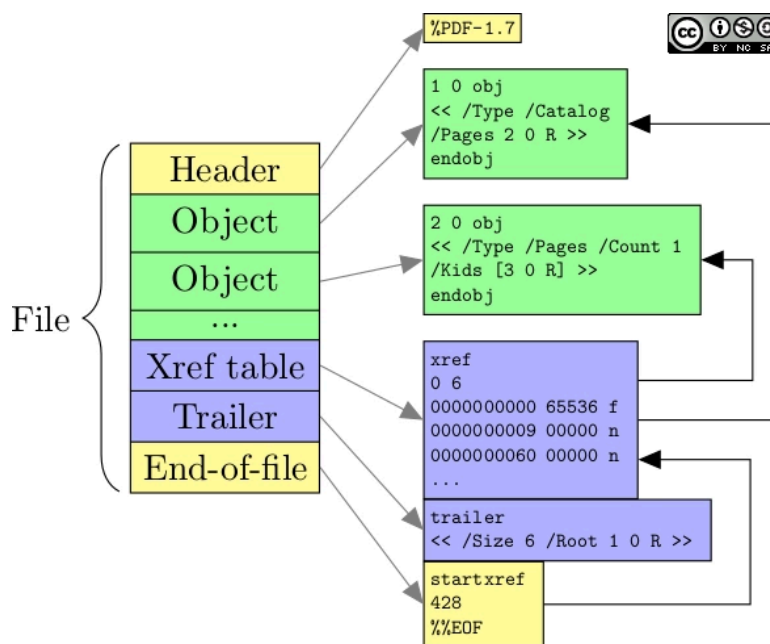


Figura 2. Estructura interna de un PDF: cabecera, objetos del cuerpo, tabla de referencias cruzadas (xref), trailer y fin de archivo. Las flechas indican qué objeto referencia a cuál. Fuente: material de la Unidad 1.

En el cuerpo los objetos forman un árbol: el *objeto de catálogo* es la raíz y apunta a los *objetos de página*, que referencian los *objetos de contenido* (instrucciones de dibujo) y los recursos (fuentes, colores). Un PDF también puede tener formularios, capas o estar encriptado.

HERRAMIENTAS

Librería	Qué hace	Cuándo se usa
PyPDF2 / pypdf	PdfReader y page.extract_text() página por página. También lee metadatos (reader.metadata: autor, creador, título).	PDFs con texto digital. pypdf es el sucesor moderno de PyPDF2.
pdfplumber	Extrae texto por página con page.extract_text()	Tablas y estructura
PyMuPDF	Lectura de PDF	Cuando se necesita velocidad y versatilidad
pdf2image + pytesseract	Convierte cada página en imagen (requiere poppler-utils) y luego aplica OCR	PDFs escaneados, sin texto digital

El archivo se abre en modo binario ('rb'). Los metadatos pueden venir vacíos (None).

Clave

Si el PDF es un escaneo, `extract_text()` no devuelve nada útil. La ruta es: PDF → imágenes (`pdf2image`) → OCR (`pytesseract`).

RELACIÓN CON LOS DEMÁS TEMAS

Cuando el PDF es complejo (columnas, tablas, fórmulas), conviene un conversor de documentos (sección 7) en lugar de `extract_text()`.

5. Texto en imágenes: OCR

IDEA CENTRAL

Si el texto está dentro de una imagen (JPG, PNG, GIF, BMP), hay que "leerlo" con OCR (Optical Character Recognition, reconocimiento óptico de caracteres).

EXPLICACIÓN

Una imagen es una grilla de píxeles (mapa de bits): no hay letras guardadas, solo puntos de color. Un OCR reconoce las formas de las letras y devuelve el texto.

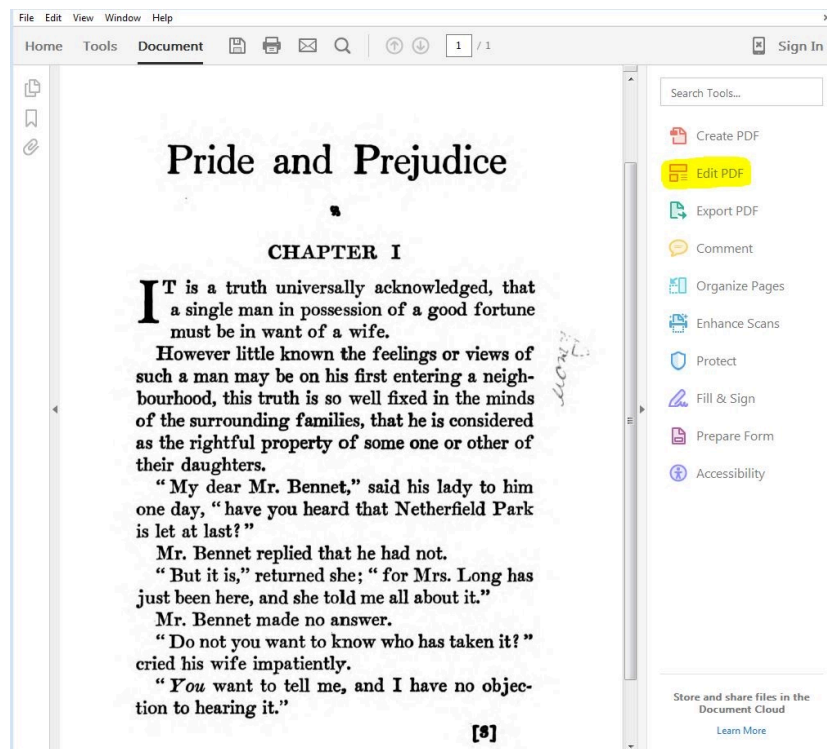


Figura 3. Página escaneada de *Pride and Prejudice* abierta en un lector de PDF. El texto existe solo como imagen, así que hace falta OCR para extraerlo. Fuente: material de la Unidad 1.

HERRAMIENTAS

- **pytesseract**: envoltorio (wrapper) de Tesseract OCR. Se le pueden indicar varios idiomas posibles: `pytesseract.image_to_string(imagen, lang="eng+spa+fra")`. Necesita Pillow para abrir la imagen y el programa `tesseract-ocr` instalado.
- **EasyOCR**: devuelve el texto junto con su ubicación en la imagen. `lector.readtext()` devuelve una lista de tuplas: (ubicación, texto, confianza). Sirve cuando importa dónde está el texto, por ejemplo en una factura.

El OCR no es 100 % preciso con ruido o distorsiones. Mejora preprocesando la imagen (binarización, suavizado, eliminación de ruido) o entrenando con un conjunto de caracteres propio.

pytesseract si solo importa el texto. EasyOCR si además importan las coordenadas de cada bloque.

6. Documentos de Word (DOCX)

DOCX es el formato predeterminado de Microsoft Word desde la versión 2007. Por dentro es XML con texto, objetos y estilos. La librería **python-docx** lee y escribe estos archivos. Para extraer el texto se recorren los párrafos:

```
doc = Document('documento.docx')
for parrafo in doc.paragraphs:
    print(parrafo.text)
```

7. Conversores de documentos

IDEA CENTRAL

Un conversor intenta reconstruir la estructura del documento (títulos, listas, tablas, orden de lectura) y la vuelca a un formato intermedio, que hoy casi siempre es Markdown.

EXPLICACIÓN

Una herramienta por formato alcanza si el archivo es simple y nos sirve un string. Un pipeline (cadena de procesamiento) que alimenta un LLM (Large Language Model, modelo de lenguaje grande) o un sistema RAG (generación aumentada por recuperación) necesita saber qué es título, qué es tabla y en qué orden se lee. `.extract_text()` pierde columnas, tablas y encabezados; un conversor intenta recuperarlos. Por eso Markdown es hoy el destino típico de la ingestión de documentos.

Herramienta	Rol	Cuándo usarla	Qué implica
MarkItDown	Envoltorio liviano a Markdown	Muchos formatos (PDF, Office, HTML, imágenes, audio). Prototipos y scripts	Instalación chica. El OCR decente pide un LLM o Azure (pago)
Docling	Parser estructurado, local	Tablas, orden de lectura, exportar a Markdown o JSON sin nube	Más pesado: descarga modelos de layout/OCR. Corre en CPU; el modo VLM es opcional
Marker	Calidad en PDF difícil	Multicolumna, fórmulas, escaneos, papers	GPU casi necesaria para el modo bueno. Código Apache 2.0; los pesos del modelo tienen licencia restrictiva
MinerU	Parser de documentos complejos	Escaneos, multicolumna, fórmulas, tablas que cruzan páginas	Pesado. El backend pipeline corre en CPU; el modo VLM pide GPU. OCR en muchos idiomas, fórmulas a LaTeX, tablas a HTML



Clave

Criterio del material en una frase: MarkItDown para entrar; Docling para estructura local; MinerU para papers y escaneos (OCR, fórmulas, tablas); Marker cuando la calidad del PDF importa y hay GPU.

No son intercambiables: MarkItDown no interpreta el layout como los otros, Docling no es el más preciso en papers densos, y MinerU y Marker son pesados para un práctico en Colab con CPU. Convertir a Markdown no reemplaza la limpieza ni la segmentación posteriores.

Docling devuelve un documento estructurado exportable (`doc.export_to_markdown()`); PyPDF2 solo devuelve un string. Marker y MinerU se usan por línea de comandos.

8. Audio y video

IDEA CENTRAL

Para sacar texto de un audio hay que hacer reconocimiento del habla (speech recognition): convertir la voz en texto.

En el material, esta sección se titula "Texto desde imágenes o video", pero su contenido trata sobre audio y video.

8.1. SpeechRecognition

SpeechRecognition abre el audio (`sr.AudioFile('audio.wav')`), lo lee con `r.record()` y lo transcribe con el servicio de Google (`r.recognize_google()`). De un video primero hay que extraer el audio con `ffmpeg`. Falla con ruido, acentos fuertes o distorsiones; ayuda eliminar ruido y normalizar el volumen.

8.2. Whisper

Whisper es un modelo de reconocimiento de voz de propósito general, entrenado con un conjunto grande y diverso de audios (680.000 horas, según el diagrama del material). Es multitarea: transcribe en varios idiomas, traduce voz al inglés, identifica el idioma y detecta actividad de voz. Es un Transformer secuencia a secuencia: recibe un espectrograma log-Mel y el decodificador predice tokens. Unos tokens especiales indican la tarea, así que un solo modelo reemplaza varias etapas de un sistema de voz tradicional.

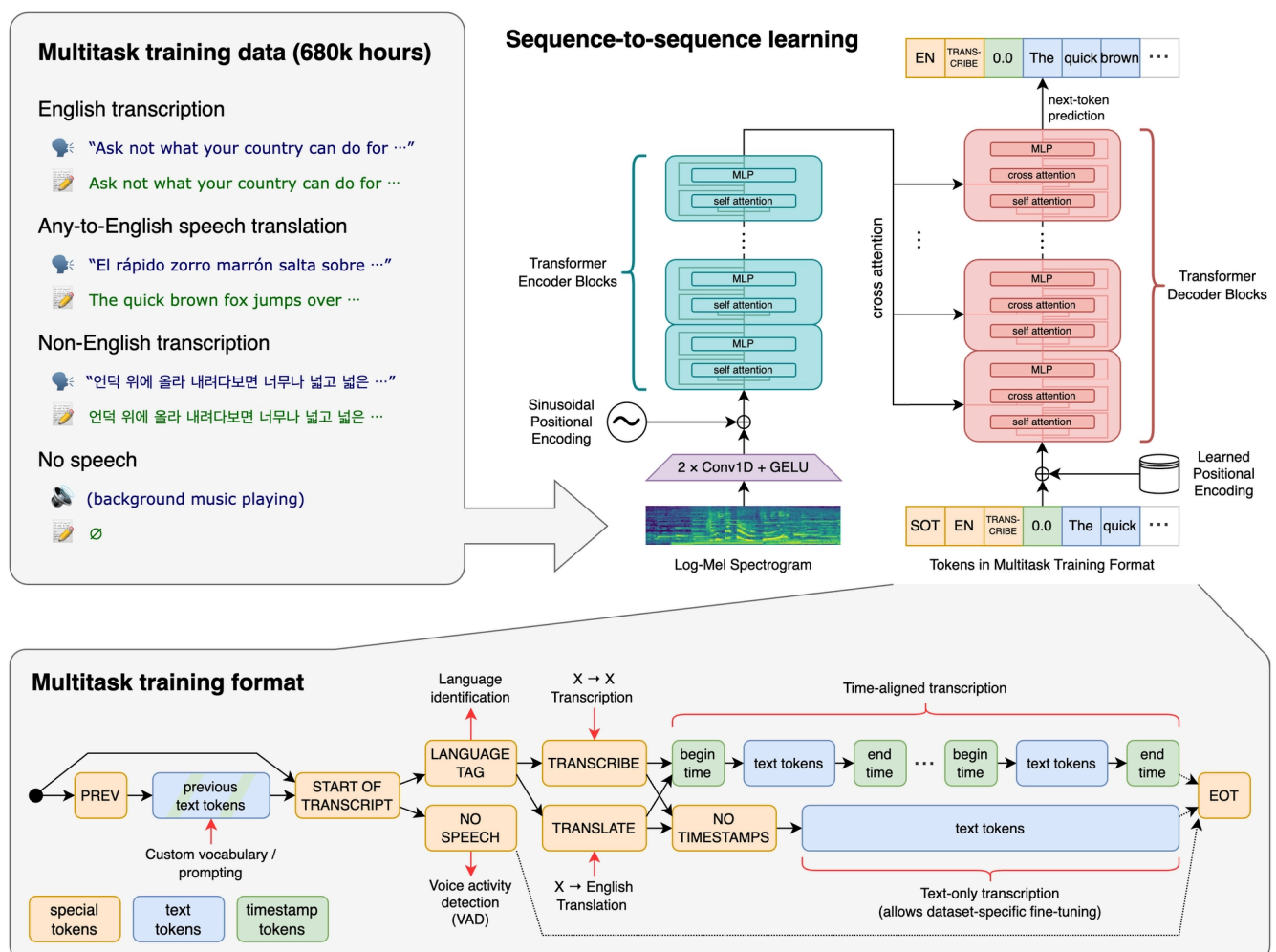


Figura 4. Arquitectura de Whisper. Arriba a la izquierda, los datos de entrenamiento multitarea; a la derecha, el Transformer secuencia a secuencia que recibe el espectrograma log-Mel; abajo, el formato de tokens especiales que indica la tarea (transcribir, traducir, con o sin marcas de tiempo). Fuente: material de la Unidad 1.

8.3. Transcripciones de YouTube

`YouTubeTranscriptApi.get_transcript(video_id)` devuelve los subtítulos como lista de diccionarios con `text`, `start` y `duration`.

CUÁNDO USAR CADA UNO

Con subtítulos disponibles, la API de YouTube es lo más directo. Para audio propio, Whisper es más completo que SpeechRecognition (traduce e identifica el idioma).

9. Otras fuentes: Wikipedia, bases de datos, tablas y JSON

9.1. Wikipedia

`wikipedia-api` pide un `user agent` y el idioma ('es'); `page.summary` da el resumen y `page.text` el contenido.

9.2. Bases de datos

Guardan texto de registros de clientes, transcripciones de llamadas, comentarios de productos o publicaciones. Librerías comunes:

`sqlite3` (incorporada en Python) para SQLite, `Psycopg2` para PostgreSQL, `PyMySQL` para MySQL y `SQLAlchemy` para varios motores con ORM (Object-Relational Mapping, mapeo objeto-relacional).

Flujo con `sqlite3`: conectar, crear un cursor, ejecutar la consulta, `fetchall()` y cerrar.

9.3. Formatos tabulares

Pandas lee Excel, CSV o Parquet (`pd.read_excel`, `pd.read_csv`) y el texto se toma de una columna.

9.4. JSON

JSON: local con `json.load(f)`; remoto con `requests.get(url)` y `json.loads(response.text)`. Queda como diccionario.

10. Web scraping

IDEA CENTRAL

El web scraping es extraer información de páginas web leyendo su HTML y pasándola a un formato estructurado (CSV, JSON o XML).

EXPLICACIÓN: CÓMO FUNCIONA LA WEB

El cliente (un navegador, o Python con `requests`) pide algo y el servidor responde. El material distingue tres escenarios:

1. **Estático:** `GET /index.html` y el servidor envía un archivo existente.
2. **Dinámico:** `GET /productos?id=123`; el servidor consulta una base de datos y genera el HTML.
3. **Formulario:** `POST /login`; el servidor verifica los datos y responde con una redirección o un error.

HERRAMIENTAS Y PATRONES

- **requests + BeautifulSoup:** se descarga la página y se buscan elementos con `soup.find('section', {'class': 'entry-content'})`, `find_all('p')` o selectores CSS (`soup.select('section.entry-content > p')`). Qué buscar depende de cada página.

- **Sesiones con login:** `requests.Session()` conserva las cookies. Se hace un POST al login con usuario y contraseña; el servidor devuelve una cookie con el ID de sesión, que se reenvía en el GET a la página protegida. Algunos sitios piden además tokens CSRF o cabeceras propias.
- **Playwright, Selenium, mechanize:** controlan un navegador real. Sirven cuando la página o el login dependen de JavaScript. Con Playwright se lanza Chromium sin ventana (`headless=True`), se navega con `page.goto(url)` y se buscan elementos con `query_selector_all('p')`. Scrapy es otra opción que menciona el material.

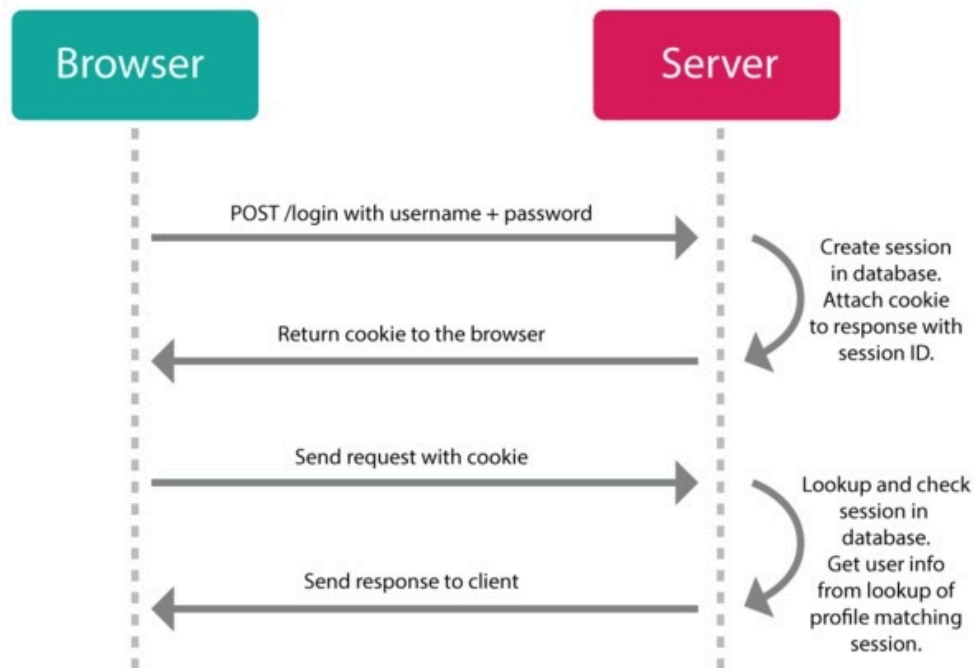


Figura 5. Inicio de sesión con cookies. El navegador envía usuario y contraseña con POST; el servidor crea la sesión y devuelve una cookie, que el cliente reenvía en cada solicitud siguiente. Fuente: material de la Unidad 1.

🔑 Clave

El scraping debe respetar los Términos de Servicio del sitio y su archivo `robots.txt`. Algunos sitios bloquean scrapers y en algunas jurisdicciones la actividad está en una zona legal gris. Si existe una API, es preferible usarla.

11. Parseo de texto

IDEA CENTRAL

Parsear es analizar un texto según reglas o patrones para separar sus partes y convertirlo en datos estructurados.

EXPLICACIÓN

[Ejemplo propio] Cuando lees "Juan, 25 años, Rosario" y lo anotás en una planilla con columnas Nombre, Edad y Ciudad, estás parseando: reconocés dónde termina cada dato y qué representa.

El material lo plantea como un puente entre el texto "que vino como vino" y un dato sobre el que se puede razonar. Después de extraer quedan restos de HTML, codificaciones raras, duplicados y enlaces. Parsear es decidir qué es mensaje y qué es ruido, sin borrar señales expresivas útiles (mayúsculas sostenidas, repeticiones, emojis). Si parte del corpus llega sucia, las conclusiones se sesgan.

DEFINICIÓN FORMAL

El parseo (parsing) descompone una cadena según reglas, patrones o gramáticas. Entra un string; se identifican delimitadores, patrones o estructuras; sale un dato estructurado (tokens, fechas, diccionarios, un árbol de sintaxis). Sirve para validar formatos, extraer emails o teléfonos y convertir datos. El web scraping también es parseo: el material extrae las noticias de Hacker News recorriendo su HTML.

TRES TÉCNICAS DEL MATERIAL

Técnica	Qué hace	Ventajas	Limitaciones
<code>split()</code> y posiciones	Divide por espacios y toma partes por índice	Simple, sin librerías	Frágil: si el formato cambia, se rompe
Expresiones regulares (re)	Busca patrones, por ejemplo <code>re.findall(patron_email, texto)</code>	Flexible, encuentra datos en cualquier posición	Patrones difíciles de leer y mantener
Librería parse	Lo inverso de <code>format()</code> : <code>parse("Hello, {}")</code> , <code>parse("Hello, John")</code>	Muy legible para formatos fijos; <code>{:d}</code> convierte a entero	Solo sirve si el formato es fijo

Ejemplo del material: el log "2023-10-01 14:30:45 ERROR: Archivo no encontrado..." se divide con `split()`; `partes[0]` es la fecha, `partes[1]` la hora, `partes[2].rstrip(':')` el nivel y el resto, unido, el mensaje.

Ejemplo con parse: `parse("{:d}-{:d}-{:d} {:d}:{:d}", "07-23-2023 16:30")` devuelve 7, 23, 2023, 16 y 30 como enteros.

Parte II. Procesamiento de texto

Segunda mitad de la Unidad 1, con los aportes de la Unidad 2 sobre stopwords. Ya tenemos el texto. Ahora hay que limpiarlo, normalizarlo y cortarlo en unidades para que un algoritmo pueda trabajar con él.

12. Limpieza y normalización

IDEA CENTRAL

Normalizar es llevar el texto a una forma uniforme para que variantes que significan lo mismo se cuenten como lo mismo.

EXPLICACIÓN

[Ejemplo propio] Para una computadora, "Café", "café" y "cafe" son tres cadenas distintas. Si contás cuántas veces se habla de café en unas reseñas, te van a salir tres conteos separados. Normalizar une esas variantes.

12.1. Minúsculas

Se usa `lower()`. Sobre una columna de Pandas: `df['tweet'].str.lower()` o `df['tweet'].apply(lambda x: x.lower())`.

12.2. Eliminación de puntuación

Quitar la puntuación reduce datos y cómputo cuando no aporta información. La expresión regular `[^\w\s]` coincide con todo lo que no sea carácter de palabra ni espacio:

```
| re.sub(r'[^\w\s]', '', "I. like. This book!") # 'I like This book'
```

El material advierte que este paso no siempre es útil. Los sistemas tipo ChatGPT, basados en LLM, trabajan con el texto original, puntuación incluida.

12.3. Eliminación de acentos

Sirve para que buscar "cafe" encuentre "café" y para homogeneizar textos de distintas personas. `unicodedata.normalize('NFKD', texto)` descompone "é" en "e" más un acento combinante, y luego se descartan los combinantes con `unicodedata.combining(c)`.

Hace falta normalizar primero porque "Ç" puede guardarse como un código (`\u00C7`) o como "C" más una cedilla combinante (`\u0043\u0327`); se ven iguales pero `"\u00C7" == "\u0043\u0327"` da `False`.

CUÁNDO CONVIENE

Útil para búsqueda y conteo. En tareas donde el acento cambia el significado (por ejemplo "papa" y "papá"), quitarlo mezcla palabras distintas. *[Ejemplo propio]*

13. Stopwords (palabras de parada)

Esta sección une lo que dicen la Unidad 1 (limpieza) y la Unidad 2 (efecto en TF-IDF).

IDEA CENTRAL

Las stopwords son palabras muy frecuentes, como "el", "la" o "de", que en muchos contextos aportan poco para distinguir un texto de otro. Quitarlas reduce el tamaño de los datos, pero hay que hacerlo con cuidado.

EXPLICACIÓN

Ejemplo del material: para la búsqueda "Cómo desarrollar un chatbot usando Python" hay muchas más páginas con "cómo" y "desarrollar" que sobre chatbots. Si el buscador ignora esas palabras comunes, se concentra en "chatbot" y "python".

DEFINICIÓN FORMAL

Las stopwords son palabras muy comunes que, en determinados contextos, no tienen significado o tienen menos que las palabras clave. Se eliminan comparando cada token contra una lista. La lista puede escribirse a mano o tomarse de NLTK: `stopwords.words('spanish')`.

Con NLTK se tokeniza con `word_tokenize` y se compara con `word.casefold()`, que ignora mayúsculas. "Esta es una introducción a NLP" queda "introducción NLP".

RELACIÓN CON TF-IDF (UNIDAD 2)

- Su TF es alto (aparecen mucho), pero su IDF es casi cero (aparecen en casi todos los documentos), así que su TF-IDF final queda bajo: TF-IDF ya las castiga.
- Igual se eliminan para reducir la dimensión, ahorrar cómputo y concentrarse en las palabras que diferencian documentos.

Clave

Frecuente no significa irrelevante. En análisis de sentimiento hay palabras "pequeñas" con mucho peso: negaciones ("no", "nada", "sin"), intensificadores ("muy", "poco", "apenas") y moduladores ("casi", "apenas", "solo"). Quitar el "no" de "no me gusta" invierte el sentido. "pero" o "si" también pueden importar al detectar matices o contradicciones. Por eso a veces se usan listas de stopwords personalizadas.

El material dice "Si eliminamos las palabras menos comunes, podemos centrarnos en las palabras clave". Por el contexto, parece querer decir "más comunes".

14. Estandarización y corrección ortográfica

14.1. Estandarización

IDEA CENTRAL

Estandarizar es convertir el texto a un formato uniforme. Incluye minúsculas, quitar puntuación, corregir ortografía, quitar stopwords y expandir abreviaturas.

El material usa un diccionario de búsqueda (`lookup_dict`) que reemplaza abreviaturas: `{'nlp': 'procesamiento del lenguaje natural', 'ia': 'inteligencia artificial'}`. La función separa en palabras, quita la puntuación de cada una, la reemplaza si está en el diccionario y vuelve a unir. "Esta es una introducción a NLP" pasa a "...introducción a procesamiento del lenguaje natural".

14.2. Corrección ortográfica

IDEA CENTRAL

Los errores de tipeo generan varias versiones de la misma palabra ("proccessing" y "processing"), que el modelo trata como palabras distintas. Corregirlos las unifica.



Clave

Las abreviaturas se resuelven antes de corregir. Si no, el corrector falla: "pq" tiene que pasar a "porque" o "por qué" antes de pasar por el corrector. Y el corrector debe ser del idioma del texto.

Librería	Uso	Resultado en el ejemplo del material (español)
TextBlob	<code>TextBlob(x).correct()</code>	Muy malo: está pensada para inglés. "Me gusta este livro" pasó a "He gust est live".
autocorrect	<code>Speller(lang='es')</code> , palabra por palabra	Corrigió "erramienta" → "herramienta", pero dejó "livro".
pyspellchecker	<code>SpellChecker(language='es')</code> , <code>.correction(palabra)</code>	Corrigió "livro" → "libro" y "abrá" → "habrá", pero cambió "Python" por "patron" y "NLP" por "nop".

Lectura de la tabla: ninguna corrige todo bien, y las tres pueden romper nombres propios o siglas.

15. Tokenización

IDEA CENTRAL

Tokenizar es dividir el texto en unidades mínimas con significado, llamadas tokens. Es un paso obligatorio para cualquier análisis.

EXPLICACIÓN

[Ejemplo propio] Es como cortar una pizza antes de repartirla: antes de contar, comparar o modelar, hay que decidir el tamaño de las porciones. Podés cortar en oraciones, en palabras o en pedazos de palabras.

DEFINICIÓN FORMAL

Hay dos tipos principales: tokenización de oraciones y tokenización de palabras. Librerías: NLTK (`word_tokenize`, `sent_tokenize`), spaCy y TextBlob (`TextBlob(t).words`).

EJEMPLO DEL MATERIAL

Para "¡Python es la mejor herramienta!":

- NLTK: ['¡Python', 'es', 'la', 'mejor', 'herramienta', '!']. Conserva el "!" como token.
- TextBlob: ['¡Python', 'es', 'la', 'mejor', 'herramienta']. Descarta la puntuación.

Las dos dejan el "¡" pegado a "Python".



Clave

Tokenizar es una decisión conceptual: elegir la unidad es elegir qué cuenta como evidencia. Las unidades posibles son oraciones (estructura del discurso), palabras (términos y coocurrencias) o subpalabras (cuando errores, neologismos y flexiones vuelven frágil la idea de palabra entera). Tokenizar no elimina nada: segmenta. Un mismo texto tokenizado de dos maneras es, para un modelo, dos realidades distintas.

Decisiones que arrastra: si la puntuación se queda, si los emojis son señal o ruido, qué hacer con tildes, mayúsculas, contracciones ("del", "al") y clíticos ("dámelo"). En reseñas importan el alcance de una negación y la repetición enfática ("bueeno").

RELACIÓN CON LOS DEMÁS TEMAS

La tokenización es la base de las stopwords, el stemming, los n-gramas y todos los vectorizadores de la Parte III. FastText (Parte III) trabaja con n-gramas de caracteres, que son una forma de tokenización en subpalabras.

16. Stemming y lematización

IDEA CENTRAL

Las dos técnicas reducen una palabra a una forma base para que "corriendo", "corre" y "correr" cuenten como lo mismo. El stemming recorta; la lematización busca la palabra real del diccionario.

EXPLICACIÓN

[Ejemplo propio] El stemming corta con tijera el final de la palabra: rápido, pero a veces deja algo que no existe ("perr"). La lematización busca en el diccionario: más lenta, pero devuelve "perro".

DEFINICIÓN FORMAL

El *stemming* (derivación) reduce las palabras a su raíz con reglas heurísticas (algoritmos Porter o Snowball) que recortan sufijos sin verificar que el resultado sea una palabra real. No usa diccionarios.

La *lematización* reduce la palabra a su lema, la forma de diccionario. Usa diccionarios lingüísticos y análisis morfológico, y tiene en cuenta el contexto para saber qué parte de la oración es la palabra. Ejemplo del material: "hojas" y "hoja" se reducen a "hoja".

EJEMPLO DEL MATERIAL: "EL PERRO ESTÁ CORRIENDO POR EL PARQUE BUSCANDO UN PESCADO."

Técnica	Herramienta	Resultado
Stemming	NLTK SnowballStemmer("spanish")	el perr esta corr por el parqu busc un pesc .
Lematización	spaCy es_core_news_sm, tok.lemma_	el perro estar correr por el parque buscar uno pescado .

Concepto	Qué hace	Cuándo se usa	Ventajas	Limitaciones
Stemming	Recorta sufijos con reglas fijas	Búsqueda y recuperación de información: encontrar una palabra en singular, plural o distintos tiempos verbales	Rápido y barato. No necesita diccionario	Produce raíces que no son palabras. Aplana matices de grado
Lematización	Busca el lema con diccionario y análisis morfológico	Cuando se necesita la palabra válida o el contexto gramatical	Lingüísticamente preciso. Usa contexto	Más lento y complejo. Necesita un modelo del idioma

🔑 Clave

Stemming = rápido, raíces que pueden no existir. Lematización = lento, lemas válidos del idioma. Las dos pueden borrar el grado (comparativo, superlativo). Si la tarea puntúa opiniones, conviene conservar el grado para no perder el matiz.

El material da como ejemplo que el stemming lleva "bueno, buenísimo, mejor" a "buen" y que la lematización lleva "buenísimo → bueno, mejor → bueno". Ver la nota al final del documento sobre este ejemplo.

17. Emojis y emoticones

IDEA CENTRAL

Los emojis (😊) y emoticones (":)") son comunes en redes sociales y transmiten tono y sentimiento. Hay que decidir qué hacer con ellos.

18.2. N-gramas

DEFINICIÓN FORMAL

Un n-grama es una secuencia contigua de n palabras. En "El gato come pescado" los bigramas son "El gato", "gato come" y "come pescado" (`nltk.util.ngrams(tokens, 2)`). Se usan para predecir la palabra siguiente, en corrección ortográfica y en traducción.

EJEMPLO DEL MATERIAL: PREDICCIÓN CON N-GRAMAS

Con un corpus de unas 20 oraciones (sol, cielo, río, montañas), el código cuenta qué palabra sigue a cada palabra (bigramas) y a cada par de palabras (trigramas). Para predecir, devuelve las 3 continuaciones más frecuentes con `Counter(...).most_common(3)`:

- "el" → cielo (2), sol (1), viento (1); "en el" → cielo (2), agua (1), firmamento (1).
- "de la" → nada: el par no aparece en el corpus. Los n-gramas solo conocen secuencias que vieron.

18.3. Co-ocurrencia

DEFINICIÓN FORMAL

Dos palabras co-ocurren si aparecen juntas dentro de un ámbito acotado. Se suele representar con una matriz: filas y columnas son palabras, cada celda cuenta cuántas veces aparece ese par. El ámbito se define de dos formas:

- **Ventana de palabras** (context window): distancia máxima. Con ventana de ± 5 , A y B co-ocurren si están a 5 palabras o menos.
- **Límites estructurales**: oración, párrafo o tweet. Si las dos palabras están en la misma unidad, co-ocurren, sin importar la distancia.

Así se capturan relaciones reales, como "inteligencia" con "artificial".

EJEMPLO DEL MATERIAL

Con un texto sobre Argentina, `CountVectorizer` por oración da una matriz X ; $X^T \cdot X$ cuenta en cuántas oraciones aparece junto cada par de palabras, y `setdiag(0)` anula la diagonal. Resulta una matriz de 91×91 donde "argentina" y "vino" valen 2.

18.4. Correlación de palabras

DEFINICIÓN FORMAL

La correlación mide fuerza y dirección de la relación entre dos variables: si la presencia de un término se asocia con la de otro en todo el corpus, contando también las ausencias. Se usa información mutua, Pearson o pruebas estadísticas. `df.corr()` de Pandas usa Pearson: 1 es correlación positiva perfecta, 0 ausencia de correlación lineal.

Ejemplo del material: 5000 documentos de 20 Newsgroups, `CountVectorizer` con las 50 palabras más frecuentes, `word_df.corr()` y un mapa de calor. Se ve correlación alta, por ejemplo, entre "file" y "program".

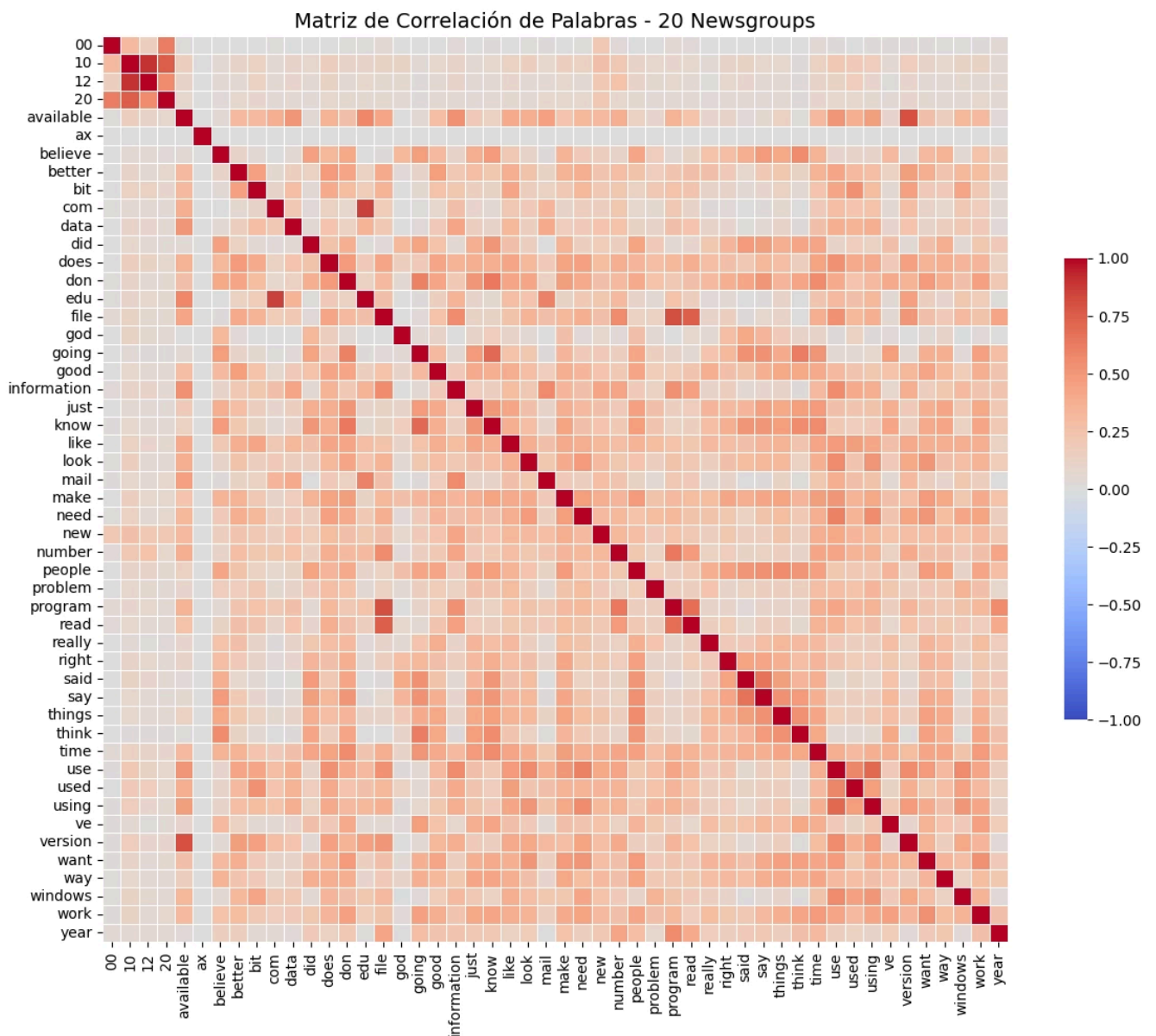


Figura 7. Matriz de correlación entre las 50 palabras más frecuentes de 5000 documentos de 20 Newsgroups. Cada celda es el coeficiente de Pearson entre dos palabras; la diagonal vale 1. Fuente: material de la Unidad 1.

Concepto	Qué hace	Ventajas	Limitaciones
Co-ocurrencia	Cuenta cuántas veces dos palabras aparecen juntas	Simple, interpretable	No distingue lo casual de lo sistemático
Correlación	Mide si los patrones de aparición de dos palabras están relacionados en todos los documentos	Considera presencia y ausencia. Indica tendencia estadística	Requiere más datos y cálculo



Clave

La co-ocurrencia es requisito para la correlación, pero no toda co-ocurrencia es una correlación significativa. Dos palabras pueden aparecer juntas por casualidad. La correlación indica una tendencia estadística consistente en el corpus.

19. Segmentación de texto (chunking)

IDEA CENTRAL

Segmentar (text chunking o splitting) es dividir un texto largo en fragmentos (chunks) más chicos y manejables: palabras, frases, oraciones o párrafos.

EXPLICACIÓN

[Ejemplo propio] Para estudiar con fichas no copiás el apunte entero en una ni ponés una palabra por ficha: cortás por temas. Un buscador semántico guarda el corpus en fragmentos y devuelve el que mejor responde la consulta.

En agentes conversacionales, los fragmentos arman el contexto del agente a partir de una base de conocimientos.

19.1. Segmentación de tamaño fijo

Se fija el tamaño de cada fragmento y, opcionalmente, el solapamiento. Es el método más común y barato. El material usa `CharacterTextSplitter(separator=".\\n")` de `langchain_text_splitters`, que corta en cada punto y aparte.

19.2. Segmentación por oraciones

El método ingenuo, `text.split(".")`, falla con abreviaturas: "Empresa S.R.L." se corta en pedazos. Opciones mejores:

Herramienta	Uso
NLTK	<code>sent_tokenize(text)</code>
spaCy	<code>[sent.text for sent in nlp(text).sents]</code>
Stanza	<code>stanza.Pipeline('es')</code> , luego <code>document.sentences</code>
pySBD	<code>pysbd.Segmenter(language="es")</code> . 22 idiomas, muy buen rendimiento en español. En el ejemplo separa bien "Casco S.R.L., que vende objetos antiguos."

19.3. Segmentación recursiva

DEFINICIÓN FORMAL

`RecursiveCharacterTextSplitter` prueba separadores en orden, `["\\n\\n", "\\n", " ", ""]` (párrafo, línea, espacio, carácter). Si un fragmento sigue superando `chunk_size`, se llama a sí mismo con el separador siguiente.

- `chunk_size`: tamaño máximo de cada fragmento.
- `chunk_overlap`: cuántos caracteres del final de un fragmento se repiten al principio del siguiente. Ese solapamiento preserva contexto y evita cortar una idea por la mitad.

EJEMPLO DEL MATERIAL, CON LOS NÚMEROS

Texto del material: "Este es un texto para analizar el comportamiento de los parámetros".

- Con `chunk_size=9`, `chunk_overlap=0`: "Este es" (7), "un texto" (8), "para" (4), "analizar" (8), "el" (2), "comporta" (8), "miento" (6)... "comportamiento" mide 14, no entra en 9, y el splitter la parte por caracteres.
- Con `chunk_size=15`, `chunk_overlap=10`: "Este es un" (10), "es un texto" (11), "un texto para" (13)... Cada fragmento repite el final del anterior.



Clave

Todos los fragmentos miden **como máximo** `chunk_size`, no exactamente `chunk_size`. El solapamiento tampoco está garantizado carácter por carácter: depende de dónde encuentre separadores el splitter, aunque en la práctica casi siempre se cumple. Un overlap muy chico da fragmentos sin sentido; uno muy grande, fragmentos enormes y repetidos.

19.4. Segmentación especializada

`MarkdownTextSplitter` corta según encabezados, listas y bloques de código. `LatexTextSplitter` respeta secciones y ecuaciones de LaTeX. Los fragmentos quedan más coherentes con la estructura.

19.5. Cómo elegir el tamaño de fragmento

Primero se preprocesa (por ejemplo, quitar HTML). Después se prueba un rango de tamaños según el contenido y los límites de tokens del modelo de embeddings. Fragmentos chicos (128 o 256 tokens) capturan información granular; grandes (512 o 1024) retienen más contexto.

19.6. Segmentación semántica

IDEA CENTRAL

Corta el texto donde cambia el tema, usando embeddings para medir qué tan parecidas son las oraciones. Los métodos anteriores cortan por tamaño o por delimitadores.

El material usa `SemanticChunker` de `chonkie` con `threshold=0.3` (umbral de similitud; puede ser 'auto'), `chunk_size=50` tokens y `similarity_window=2` (compara con las 2 oraciones anteriores). Con dos oraciones sobre IA y dos sobre cambio climático, devuelve dos fragmentos, uno por tema.

Método	Corta por	Cuándo se usa	Ventajas	Limitaciones
Tamaño fijo	Separador o cantidad de tokens	Opción por defecto, textos homogéneos	Barato, sin librerías de PLN	Puede cortar ideas por la mitad
Por oraciones	Límites de oración	Modelos que trabajan a nivel de oración	Conserva el contexto de cada oración	Oraciones de largo muy variable
Recursiva	Separadores jerárquicos hasta llegar al tamaño	Texto general para RAG	Respeto párrafos cuando puede. Control con overlap	Sigue sin entender el significado
Especializada	Sintaxis de Markdown o LaTeX	Documentación, papers	Fragmentos coherentes con la estructura	Solo para esos formatos
Semántica	Cambio de tema medido con embeddings	RAG, recuperación, análisis de documentos	Agrupar contenido relacionado	Necesita un modelo de embeddings. Más costosa

RELACIÓN CON LOS DEMÁS TEMAS

La segmentación semántica usa embeddings y similitud de coseno, que se explican en la Parte III. Vale la pena releer esta sección después de estudiar esa parte.

Parte III. Representación vectorial de texto

Corresponde a la Unidad 2. Los algoritmos de aprendizaje automático trabajan con números. Esta parte explica cómo convertir palabras, oraciones y documentos en vectores, desde los métodos de conteo hasta los embeddings.

20. Por qué convertir texto en vectores

Al convertir el texto en vectores numéricos podemos aplicarle algoritmos de aprendizaje automático: Naive Bayes, regresión logística, SVM (Support Vector Machines, máquinas de vectores de soporte) o redes neuronales. La unidad recorre dos familias:

- **Codificaciones** (one-hot, conteo, TF-IDF, hash): cuentan o marcan palabras. Dan vectores dispersos (la mayoría ceros) y no capturan significado.
- **Embeddings** (Word2Vec, FastText, BERT, Sentence-BERT): se aprenden de los datos. Dan vectores densos en los que palabras de significado parecido quedan cerca.

El *corpus* es el conjunto de documentos y el *vocabulario*, la lista de palabras distintas del corpus.

21. One-hot encoding

IDEA CENTRAL

Cada palabra se representa con un vector del tamaño del vocabulario, con un 1 en la posición de esa palabra y 0 en todas las demás.

EXPLICACIÓN

[Ejemplo propio] Es como una fila de casilleros, uno por cada palabra posible. Para representar "perro" encendés solo su casillero. Dos palabras distintas nunca comparten casillero encendido, así que la representación no dice nada sobre si se parecen.

DEFINICIÓN FORMAL

One-hot convierte variables categóricas (sin orden, como nombres de ciudades) en vectores binarios. En PLN, cada palabra es un vector de n dimensiones, con n = tamaño del vocabulario. Con ['gato', 'perro', 'casa']: gato = [1, 0, 0], perro = [0, 1, 0], casa = [0, 0, 1].

Se implementa con `OneHotEncoder` de `scikit-learn` o con `pd.get_dummies()` de `Pandas`.

LIMITACIONES

- Con vocabularios grandes los vectores tienen muchísimas dimensiones.
- No tiene en cuenta la similitud semántica. Con $V = \{\text{Ten, un, buen, gran, día}\}$, "buen" = [0,0,1,0,0] y "gran" = [0,0,0,1,0] quedan tan lejos como "día" y "ten", aunque "buen día" y "gran día" signifiquen casi lo mismo.



Clave

En one-hot todas las palabras son ortogonales: la similitud de coseno entre dos palabras distintas siempre es 0. Por eso no sirve para medir parecido de significado.

22. Count Vectorizer (vectorización por conteo)

IDEA CENTRAL

Cada documento se representa con un vector que cuenta cuántas veces aparece cada palabra del vocabulario en ese documento.

EXPLICACIÓN

[Ejemplo propio] Es como el ticket del súper: dice cuántas unidades de cada producto llevaste, pero no en qué orden los pusiste en el carrito. Por eso a este enfoque también se lo llama bolsa de palabras.

EJEMPLO DEL MATERIAL

	Me	gusta	jugar	fútbol	el	tenis
Doc 1: "Me gusta jugar fútbol"	1	1	1	1	0	0
Doc 2: "Me gusta el tenis"	1	1	0	0	1	1

Con scikit-learn: `CountVectorizer().fit_transform(corpus)`, que por defecto pasa todo a minúsculas. "El perro está en el jardín" tiene un 2 en la columna "el".

VENTAJAS Y LIMITACIONES

Es fácil de entender e implementar. No tiene en cuenta el orden de las palabras. Además, da mucho peso a palabras muy frecuentes como "el" o "la", que aportan poco para clasificar documentos. TF-IDF resuelve este último problema.

23. TF-IDF

IDEA CENTRAL

TF-IDF pondera cada palabra: sube si la palabra aparece mucho en el documento y baja si aparece en muchos documentos del corpus.

EXPLICACIÓN

[Ejemplo propio] En apuntes de PLN, "texto" aparece en todas las páginas y no identifica ninguna. "Lematización" aparece mucho en una sola sección. TF-IDF le da poco peso a la primera y mucho a la segunda.

DEFINICIÓN FORMAL

TF-IDF significa Term Frequency - Inverse Document Frequency (frecuencia de término, frecuencia inversa de documento). Mide qué tan importante es una palabra para un documento dentro de un corpus. Tiene dos componentes.

FÓRMULAS

$$\text{TF} \left(t, d \right) = \frac{\text{número de veces que } t \text{ aparece en } d}{\text{número total de términos en } d}$$
$$\text{IDF} \left(t, D \right) = \log \left(\frac{\text{número total de documentos en } D}{\text{número de documentos que contienen } t} \right)$$
$$\text{TF-IDF} \left(t, d, D \right) = \text{TF} \left(t, d \right) \times \text{IDF} \left(t, D \right)$$

- t : un término (una palabra).
- d : un documento del corpus (un artículo, un tweet, un post).
- D : el corpus completo.

- TF: frecuencia relativa de la palabra en el documento.
- IDF: logaritmo del cociente entre el total de documentos y los documentos que contienen la palabra. Si la palabra está en todos, el cociente es 1 y el log es 0.

El logaritmo amortigua el efecto de la frecuencia. Sin él, las palabras muy raras tendrían un peso desproporcionado; con él, el IDF crece de forma gradual a medida que el término se vuelve más raro.

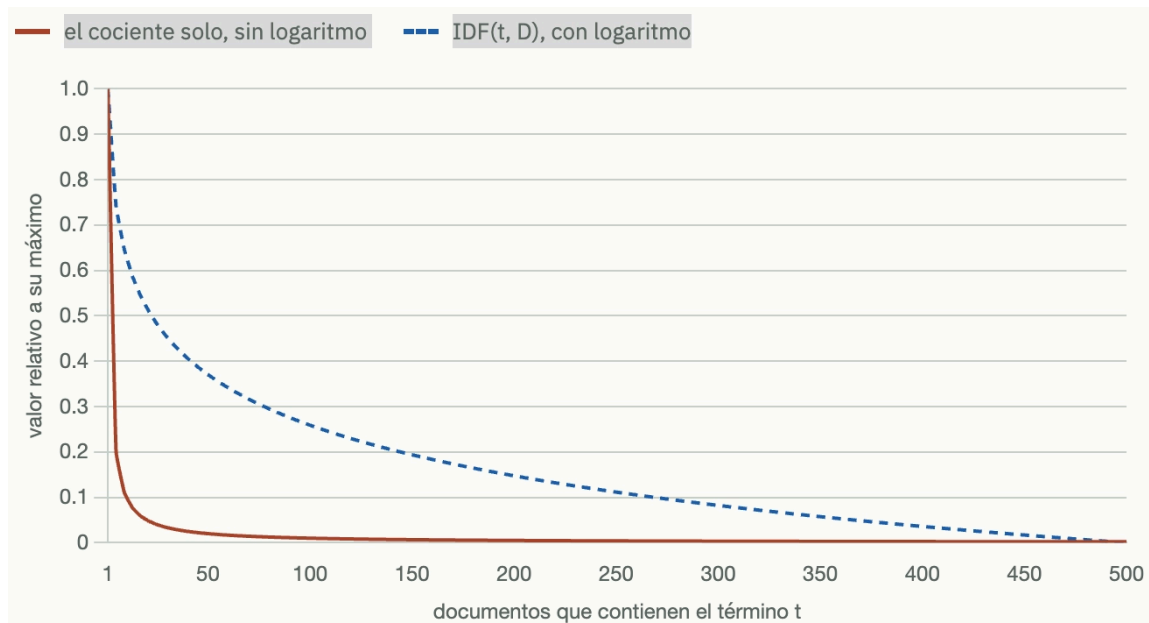


Figura 8. Valor del IDF según la cantidad de documentos que contienen el término, con y sin logaritmo (valores relativos a su máximo). El cociente sin logaritmo cae de golpe en los primeros documentos; con logaritmo la curva baja de forma gradual. Fuente: material de la Unidad 2.

CÁLCULO NUMÉRICO

[Ejemplo propio] Corpus de 3 documentos. El documento d = "gato come gato pescado" tiene 4 términos. "gato" aparece 2 veces en d y solo en ese documento. Uso logaritmo natural.

- $TF(\text{gato}, d) = 2 / 4 = 0,5$
- $IDF(\text{gato}, D) = \ln(3 / 1) = \ln 3 \approx 1,099$
- $TF\text{-}IDF = 0,5 \times 1,099 \approx 0,549$

Si "gato" apareciera en 2 de los 3 documentos: $IDF = \ln(3/2) \approx 0,405$ y $TF\text{-}IDF = 0,5 \times 0,405 \approx 0,203$. La misma palabra pesa menos porque es más común en el corpus.

EJEMPLO DEL MATERIAL: TfidfVectorizer de SCIKIT-LEARN

Corpus: "Juan tiene algunos gatos", "Los gatos comen pescado", "Yo comí una gran hamburguesa". Hay 12 palabras, así que cada documento es un vector de 12 dimensiones. scikit-learn devuelve $IDF = 1,693$ para las palabras presentes en 1 documento y 1,288 para "gatos" (presente en 2). Usa una versión suavizada del IDF:

$$IDF = \ln \left(\frac{1 + N}{1 + df(t)} \right) + 1$$

N es la cantidad de documentos y $df(t)$ la cantidad de documentos que contienen t . Cuenta del material para una palabra presente en 1 documento: $\ln((1+3)/(1+1)) + 1 = \ln 2 + 1 \approx 0,693 + 1 = 1,693$. Para "gatos": $\ln(4/3) + 1 \approx 1,288$.

La matriz del material muestra 0,5286 y 0,4020 para el primer documento. [Explicación propia] Esos valores salen de normalizar el vector a largo 1 (norma L2), que scikit-learn hace por defecto. El vector sin normalizar es [1,693; 1,693; 1,693; 1,288]. Su norma es $\sqrt{3 \times 1,693^2 + 1,288^2} \approx 3,203$. Dividiendo: $1,693 / 3,203 \approx 0,5286$ y $1,288 /$

$3,203 \approx 0,4020$. En el tercer documento las 5 palabras tienen el mismo IDF, así que cada una vale $1/\sqrt{5} \approx 0,4472$.

Clave

Si una palabra aparece muchas veces en un documento, su TF-IDF sube. Si aparece en muchos documentos del corpus, su TF-IDF baja. Una palabra presente en todos los documentos tiene $IDF = \log(1) = 0$ en la fórmula básica.

RELACIÓN CON LOS DEMÁS TEMAS

TF-IDF se usa en recuperación de información, clasificación y agrupamiento de documentos. Mejora a CountVectorizer porque descuenta las palabras comunes, pero sigue sin capturar significado: "auto" y "coche" son columnas distintas sin relación.

24. Vectorización de hash

IDEA CENTRAL

En lugar de guardar un vocabulario, se pasa cada palabra por una función de hash que decide en qué posición del vector se cuenta. El tamaño del vector se fija de antemano.

EXPLICACIÓN

[Ejemplo propio] Un guardarropas con 10 percheros y una regla que, según las letras del nombre, dice dónde colgar cada abrigo. No hace falta lista de clientes, pero a veces dos abrigos caen en el mismo perchero: una colisión.

DEFINICIÓN FORMAL

Una función de hash toma una entrada y devuelve un valor de longitud fija. Que dos entradas distintas den el mismo valor se llama *colisión de hash*. MD5 o SHA-256 se usan en criptografía; para vectorizar se prefieren algoritmos no criptográficos como MurmurHash, por eficiencia.

Proceso: tokenizar, calcular el hash de cada palabra y usarlo como índice en una matriz dispersa (sparse matrix, mayoría de ceros) de (documentos) $\times$ (n_features). Con HashingVectorizer(n_features=10), 8 frases dan una matriz de 8×10 . Los valores se normalizan entre -1 y 1 salvo con norm=None.

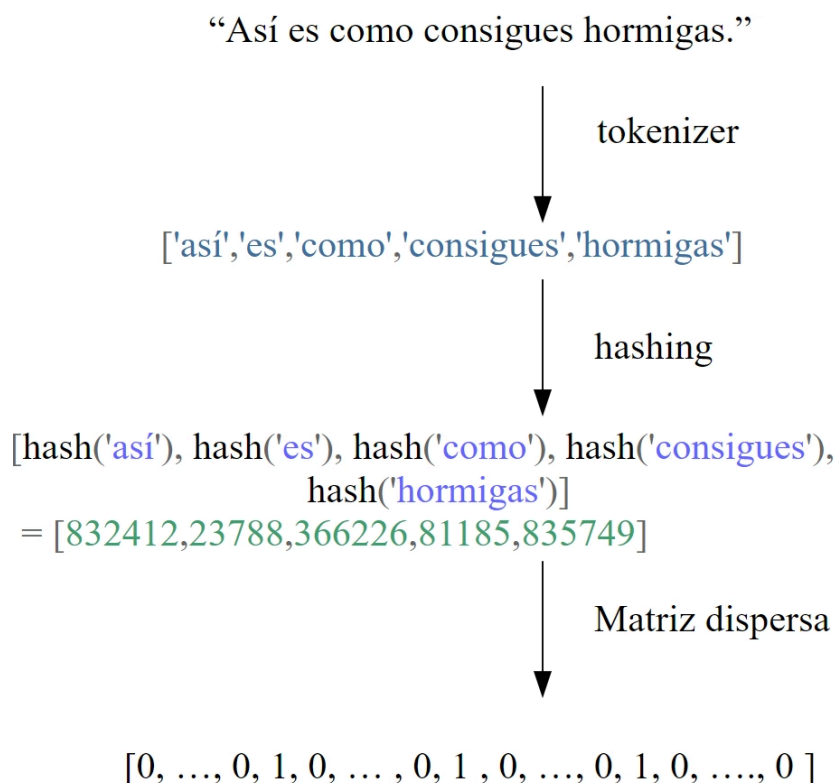


Figura 9. Vectorización de hash: la oración se tokeniza, cada token pasa por la función de hash y el resultado indica qué posición de la matriz dispersa se incrementa. Fuente: material de la Unidad 2.

VENTAJAS Y LIMITACIONES

Es una técnica sin estado: no guarda vocabulario, ahorra memoria y da vectores de tamaño fijo. A cambio, no permite volver de una columna a la palabra, lo que dificulta interpretar. Con suficientes `n_features`, las colisiones suelen ser un problema menor.

25. Comparación de las codificaciones

Concepto	Qué hace	Aplica a	Ventajas	Limitaciones
One-hot	Un 1 en la posición de la palabra y 0 en el resto	Palabras	Fácil de entender e implementar	Alta dimensión. Palabras ortogonales, sin similitud
Count Vectorizer	Cuenta cada palabra en el documento	Oraciones o documentos	Considera la frecuencia	Ignora el orden. Sobrepondera palabras comunes
TF-IDF	Conteo ponderado por rareza en el corpus	Oraciones o documentos	Considera frecuencia y relevancia	Ignora el orden y el significado
Hash Vectorizer	Cada palabra va a una posición según su hash	Palabras, oraciones o documentos	Tamaño fijo sin importar el vocabulario. Ahorra memoria	No se puede volver a la palabra. Colisiones

Clave

Estas codificaciones representan el texto con números, pero no capturan semántica ni contexto: tratan cada palabra como una entidad aislada. El coseno entre dos vectores TF-IDF mide cuántas palabras comparten (ponderadas), no si dos palabras distintas significan algo parecido. Para eso existen los embeddings.

26. Word embeddings

IDEA CENTRAL

Un embedding (incrustación) de palabra es un vector denso aprendido de los datos, en el que palabras de significado parecido quedan cerca en el espacio.

EXPLICACIÓN: RASGOS SEMÁNTICOS

El material usa un ejemplo de la Universidad Carnegie Mellon. Tomemos "man", "woman", "boy" y "girl". Podemos ubicarlas en un plano con dos ejes: género (x) y edad (y). Esos ejes son *rasgos semánticos*: representan parte del significado de cada palabra.

Palabra	Género	Edad	Realeza
man	1	7	1
woman	9	7	1
boy	1	2	1
girl	9	2	1
king	1	8	8
queen	9	7	8
prince	1	2	8
princess	9	2	8

Con dos rasgos, "man" y "king" quedarían en el mismo lugar; por eso se agrega "realeza" y se pasa a 3 dimensiones. Cada palabra queda descrita por un *vector de características*. Los valores no tienen que ser simétricos ("king" tiene edad 8 y "queen" 7).

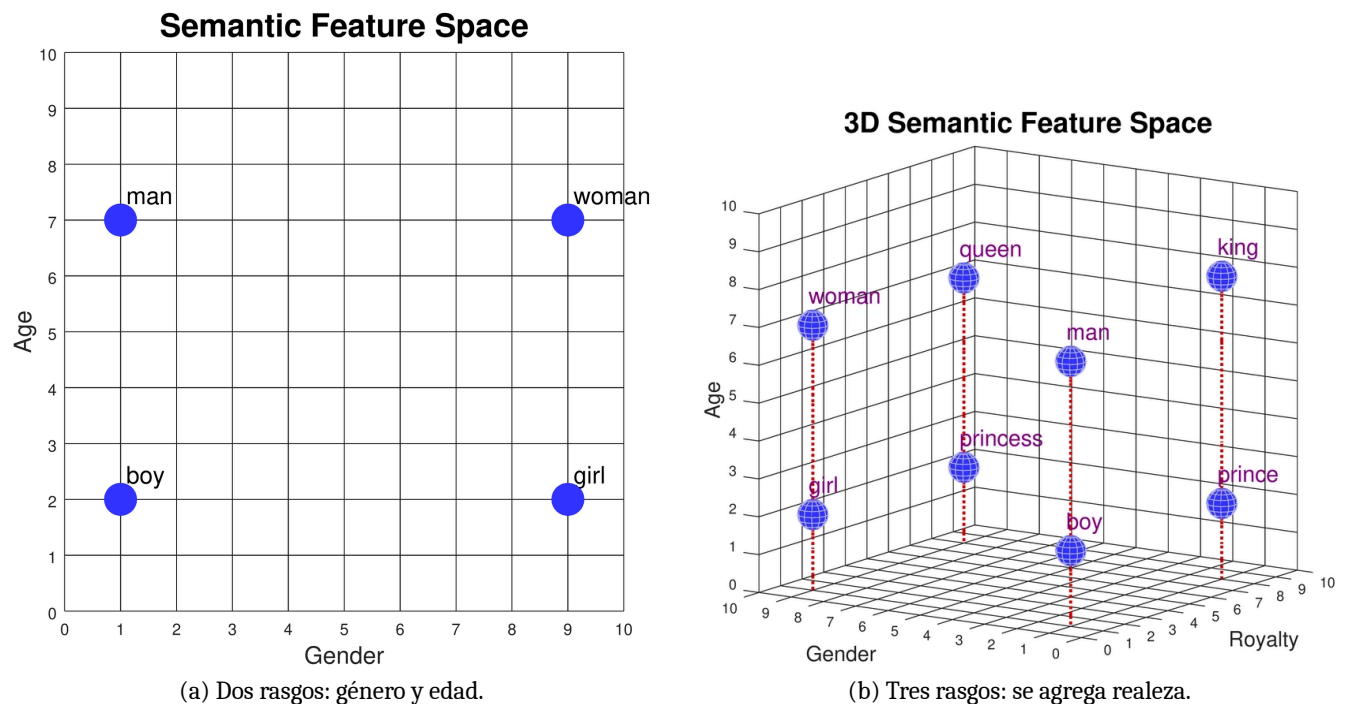


Figura 10. Espacio de rasgos semánticos del ejemplo de Carnegie Mellon. Con dos rasgos, palabras como "man" y "king" quedarían en el mismo punto; el tercer rasgo, realeza, las separa. Fuente: material de la Unidad 2.

[Explicación propia] Un embedding real tiene cientos de dimensiones que aprende un algoritmo, así que no se pueden interpretar una por una.

DEFINICIÓN FORMAL

Los word embeddings representan la semántica de las palabras como vectores. Se basan en que palabras que aparecen en contextos similares tienen significados similares: "perro" y "gato" aparecen en "mi ____ come mucho alimento", así que sus vectores quedan cerca. Word2Vec o GloVe los aprenden con redes neuronales sobre grandes corpus, y capturan relaciones como "rey" es a "hombre" lo que "reina" es a "mujer".

Representación	Tipo de vector	Dimensión	Cómo se obtiene
One-hot o hash	Disperso (casi todo ceros)	Alta	Codificado de forma rígida
Word embedding	Denso (casi todos los valores distintos de cero)	Relativamente baja	Aprendido a partir de los datos

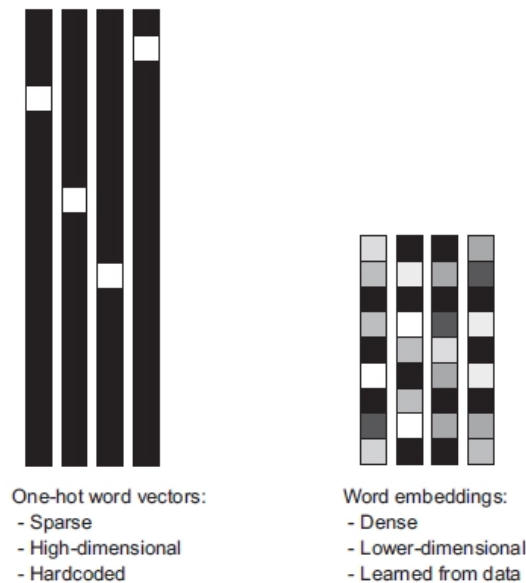


Figura 11. Comparación visual entre vectores one-hot (dispersos, de alta dimensión, codificados a mano) y word embeddings (densos, de menor dimensión, aprendidos de los datos). Fuente: material de la Unidad 2.

27. Similitud de coseno

IDEA CENTRAL

La similitud de coseno mide el ángulo entre dos vectores. Si apuntan en la misma dirección, el valor es cercano a 1 y los consideramos parecidos.

EXPLICACIÓN

[Ejemplo propio] Dos personas señalan con el brazo. Si señalan al mismo lugar, coinciden aunque una tenga el brazo más largo. El coseno mira hacia dónde apunta el vector y no cuánto mide.

FÓRMULA

$$\cos \left(\theta \right) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

- A, B : los dos vectores. A_i es su componente i .
- θ : ángulo entre ellos.
- $A \cdot B$: producto escalar (multiplicar componente a componente y sumar).
- $\|A\|, \|B\|$: normas o magnitudes (raíz de la suma de los cuadrados).

Interpretación: 1 = misma dirección (ángulo 0°), 0 = ortogonales, sin relación (90°), -1 = opuestos (180°).

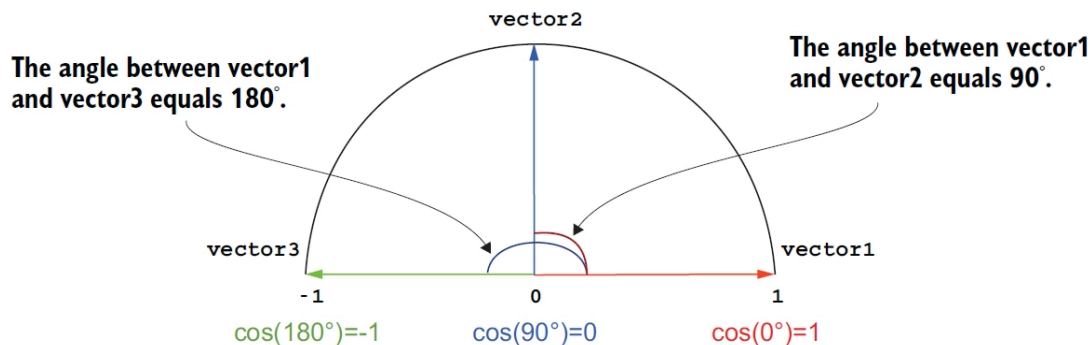


Figura 12. Valores del coseno según el ángulo entre dos vectores: 1 a 0°, 0 a 90° y -1 a 180°. Fuente: material de la Unidad 2.

CÁLCULO NUMÉRICO (EJEMPLO DEL MATERIAL)

Cat = [48, 12], Dog = [50, 10].

- Producto escalar: $50 \times 48 + 10 \times 12 = 2400 + 120 = 2520$
- $\|Cat\| = \sqrt{48^2 + 12^2} = \sqrt{2448} \approx 49,5$
- $\|Dog\| = \sqrt{50^2 + 10^2} = \sqrt{2600} \approx 51,0$
- $\cos(\theta) = 2520 / (51,0 \times 49,5) \approx 0,998$. Ángulo $\approx 2,7^\circ$. Son muy similares.

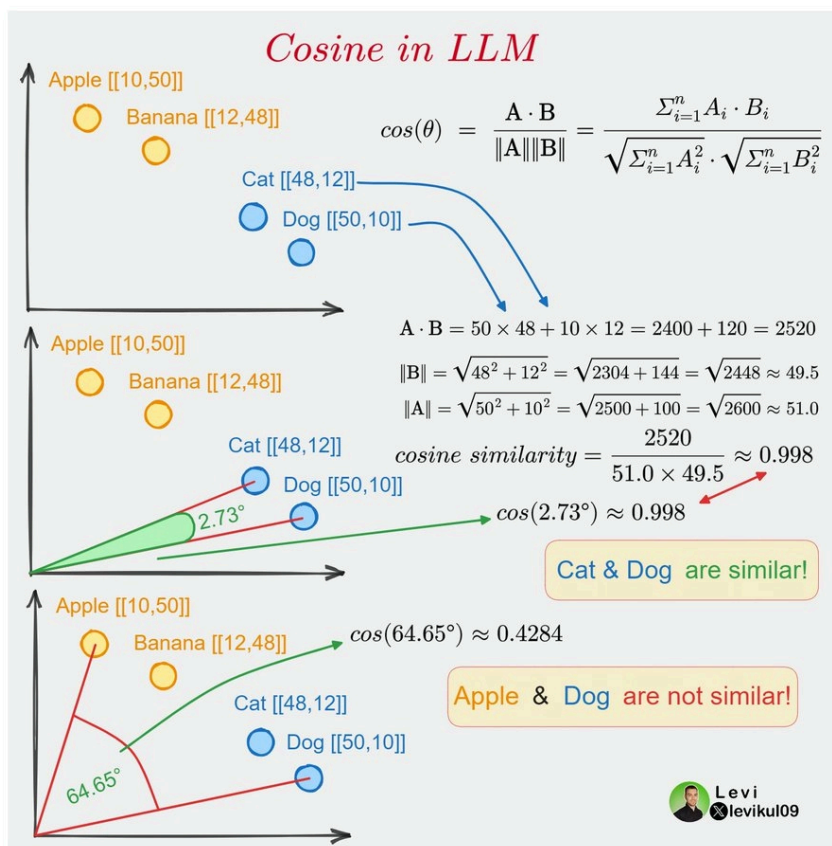


Figura 13. Similitud de coseno entre vectores de dos dimensiones. Panel superior: los cuatro vectores. Panel central: Cat y Dog, con coseno 0,998 (similares). Panel inferior: un par no similar; el gráfico lo rotula como Apple y Dog, pero el valor corresponde a Apple y Cat (ver Observaciones sobre el material). Fuente: material de la Unidad 2.

Segundo ejemplo del material con scikit-learn (cosine_similarity): consulta [0,3; 0,5; 0,5] contra Banana [0,1; 0,2; 0,4] = 0,9375, Manzana [0,3; 0,4; 0,5] = 0,9942 y Mandarina [0,1; 0,3; 0,1] = 0,9028. La más parecida es Manzana.

POR QUÉ SE PREFIERE A LA DISTANCIA EUCLIDIANA

El coseno solo mira la dirección: un documento corto y uno largo sobre el mismo tema quedan similares, y la distancia euclidiana los separaría por su magnitud. En alta dimensión (300 en GloVe), la "maldición de la dimensionalidad" le quita sentido a la distancia euclidiana. Además tiene una escala intuitiva (-1 a 1).

Clave

Similitud de coseno = producto escalar dividido por el producto de las normas. Vale 1 si los vectores apuntan igual, 0 si son ortogonales y -1 si son opuestos. Es la medida estándar de parecido entre embeddings.

28. Tipos de modelos de embeddings

IDEA CENTRAL

Los embeddings se dividen según si la misma palabra tiene siempre el mismo vector (independientes del contexto) o si el vector cambia según la oración (dependientes del contexto).

Tipo	Qué hace	Ejemplos	Limitación
Independiente del contexto ("embeddings clásicos")	Un vector por palabra, aprendido con redes neuronales superficiales o factorizando matrices de co-ocurrencia. Se distribuyen preentrenados	Word2Vec, GloVe, FastText	No distingue sentidos de una palabra polisémica
Dependiente del contexto	Vectores distintos para la misma palabra según el contexto. "banco" tiene un vector en una oración de finanzas y otro en una de deporte	Basados en RNN (redes neuronales recurrentes): CoVe, Flair, ELMo. Basados en Transformers: BERT, ALBERT	Más costosos de calcular

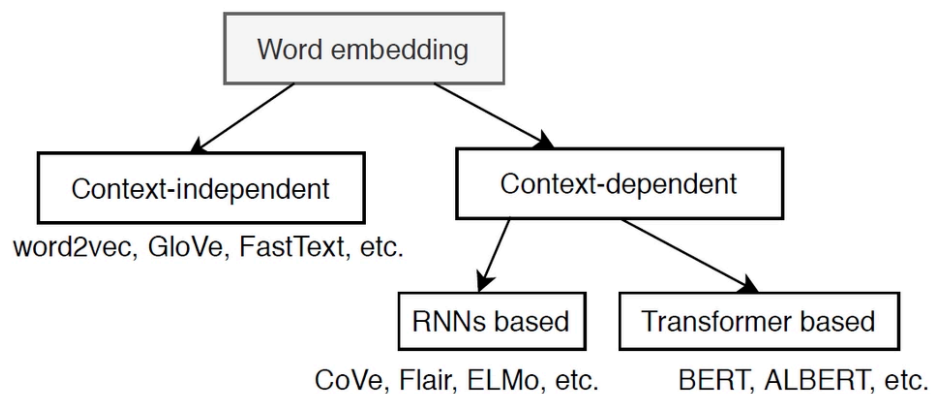


Figura 14. Taxonomía de los word embeddings: independientes del contexto (Word2Vec, GloVe, FastText) y dependientes del contexto, basados en RNN (CoVe, Flair, ELMo) o en Transformers (BERT, ALBERT). Fuente: material de la Unidad 2.

29. Word2Vec: Skip-gram y CBOW

IDEA CENTRAL

Word2Vec (2013) aprende embeddings entrenando una red neuronal simple a predecir palabras a partir de su contexto, o el contexto a partir de una palabra. Los pesos de la red terminan siendo los vectores.

EXPLICACIÓN

Word2Vec mostró que se pueden aprender eficientemente vectores densos donde palabras parecidas quedan cerca. Tiene dos algoritmos de entrenamiento:

Concepto	Qué predice	Ejemplo
Skip-gram	Las palabras de contexto a partir de la palabra central	De "coche" predecir "neumático", "carretera", "vehículo", "puerta"
CBOW (Continuous Bag of Words, bolsa de palabras continua)	La palabra central a partir de las de contexto	De "El", "gato", "al", "ratón" predecir "persigue"

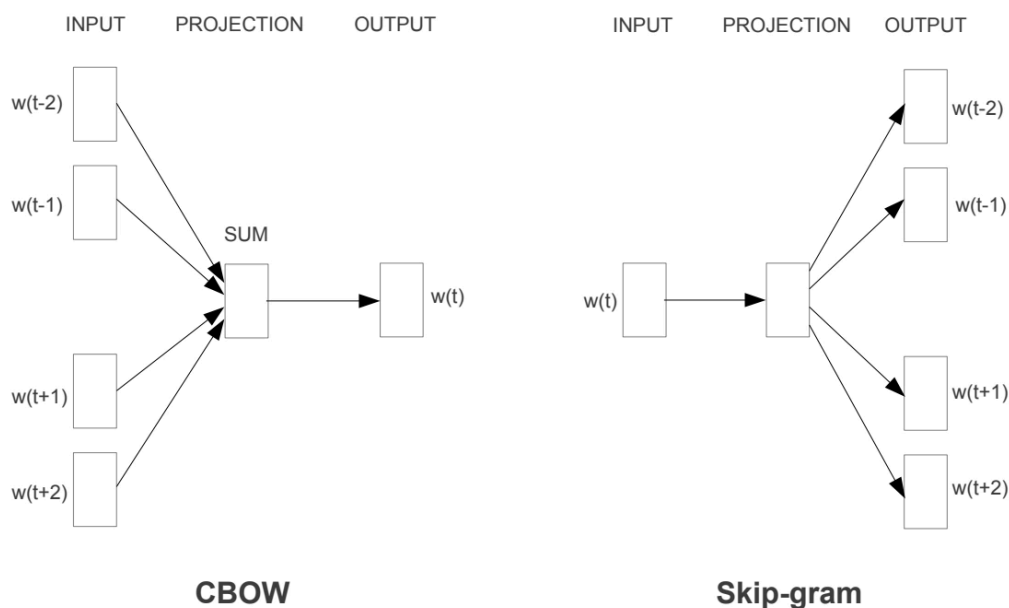


Figura 15. Los dos algoritmos de Word2Vec. CBOW suma el contexto $w(t-2) \dots w(t+2)$ para predecir la palabra central $w(t)$; Skip-gram parte de $w(t)$ para predecir su contexto. Fuente: material de la Unidad 2.

29.1. Skip-gram

La palabra central entra como vector one-hot y la red se optimiza para dar valores cercanos a 1 en las palabras de contexto. El tamaño de la ventana (window size) es un hiperparámetro: en "A quick brown fox", con ventana 1, para "quick" se predicen "A" y "brown".

La capa oculta es una matriz de (tamaño del vocabulario) $\times$ (neuronas): en el ejemplo del material, 10.000×300 . [Cálculo propio] Son 3.000.000 de pesos, y cada fila es el vector de 300 dimensiones de una palabra. 300 es lo que usó Google con Google News; es un hiperparámetro.

Si dos palabras tienen contextos parecidos ("lluvia" y "clima"), el modelo produce resultados parecidos para ellas y sus vectores quedan cerca.

Skip-gram no distingue si una palabra de contexto está a la izquierda o a la derecha: en "El gato maúlla" no sabe si "gato" va antes o después de "maúlla". Sus resultados reflejan coocurrencias y no alcanzan para tareas que dependen del orden o la gramática.

29.2. CBOW

Las C palabras de contexto entran como one-hot de tamaño V , pasan por la matriz W ($V \times N$) y se promedian en una capa oculta de N neuronas; W' ($N \times V$) lleva a una salida softmax de largo V . Al terminar, los pesos de la capa oculta son los vectores, de dimensión N .

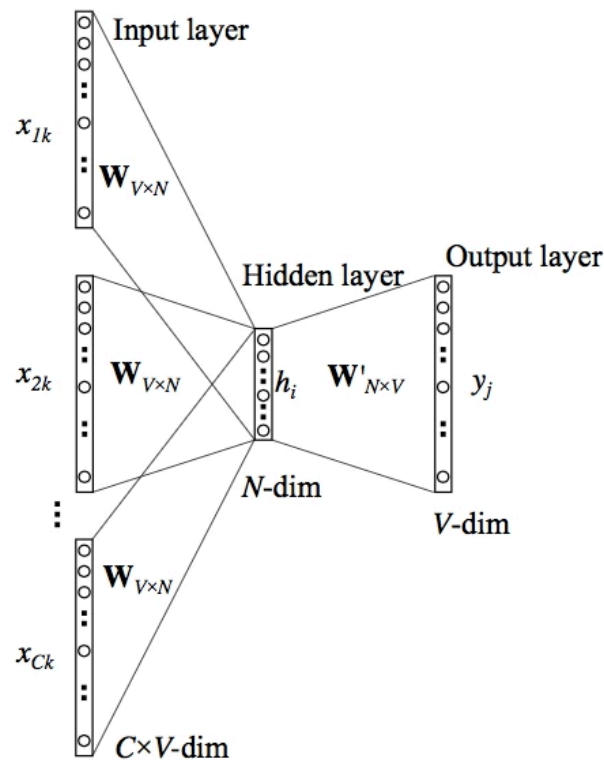


Figura 16. Arquitectura de CBOW: C vectores one-hot de tamaño V entran por la matriz W ($V \times N$), se promedian en la capa oculta de N neuronas y la matriz W' ($N \times V$) produce la salida de tamaño V. Fuente: material de la Unidad 2.

EN CÓDIGO (GENSIM)

- El material carga con `KeyedVectors.load_word2vec_format(...)` un modelo Skip-gram en español de 1.000.653 palabras y 300 dimensiones.
- `model.similarity('perro', 'conejo') = 0,611`. `most_similar(positive=['gato'])`: perro (0,745), zorro (0,706), conejo (0,702)...
- Entrenar: `Word2Vec(sentences, vector_size=100, window=4, min_count=1, sg=0)`. Con contexto ["el", "gato", "come"], la palabra más probable es "pescado".

ANALOGÍAS

`most_similar(positive=['mujer', 'rey'], negative=['hombre'])` suma los vectores de positive, resta los de negative y busca por coseno la palabra más cercana: "reina" (0,749).

Clave

Skip-gram: palabra central $\rightarrow$ contexto. CBOW: contexto $\rightarrow$ palabra central. En gensim se elige con `sg` (1 = Skip-gram, 0 = CBOW). Los dos producen un vector fijo por palabra y capturan analogías como `rey - hombre + mujer  $\approx$  reina`.

30. FastText

IDEA CENTRAL

FastText (Bojanowski et al., FAIR, 2016) extiende Word2Vec: aprende vectores para n-gramas de caracteres y representa cada palabra como la suma de sus n-gramas.

EXPLICACIÓN

Con $n = 3$, "donde" se escribe <donde> (los signos marcan inicio y fin) y se descompone en <do, don, ond, nde, de>.

Característica	Qué significa
Morfología	Palabras de la misma familia comparten n-gramas y quedan cerca
Palabras fuera de vocabulario (OOV, out of vocabulary)	Una palabra no vista igual obtiene vector
Sigue siendo estático	Un único vector por palabra: no distingue "banco" (dinero) de "banco" (asiento)

EJEMPLO DEL MATERIAL

FastText(sentences, vector_size=50, window=3, min_count=1, epochs=400, min_n=3, max_n=6) sobre 4 oraciones. min_n y max_n son el largo mínimo y máximo de los n-gramas. min_count=1 evita que el corpus diminuto quede sin vocabulario (el valor por defecto es 5).

- Vecinos de "gato": gatos (0,666) y gatito (0,491).
- "gatitos" no está en el vocabulario, pero recibe vector: gato/gatitos = 0,280.

Para el español, con mucha flexión, eso evita necesitar un vector por cada forma.

31. Línea temporal y embeddings contextuales (BERT)

31.1. Hitos en embeddings

Año	Modelo	Qué cambió
2013	Word2Vec	Embeddings densos y analogías
2014	GloVe	Como Word2Vec, entrenado con co-ocurrencia global (Stanford)
2014	Doc2Vec	Un vector por documento o párrafo
2016	FastText	N-gramas de caracteres: morfología y OOV
2017	InferSent	Embeddings de oración supervisados (NLI)
2018	ELMo	Primer embedding contextual masivo (biLSTM)
2018	BERT	Contextual con Transformers
2018	USE	Universal Sentence Encoder (Google), multitarea
2019	Sentence-BERT	Embeddings de oración comparables con coseno
2022	MTEB	Benchmark para elegir modelo de embedding
2025	EmbeddingGemma	Embeddings compactos para el dispositivo (Gemma)

Según el material, GloVe solo cambia el entrenamiento respecto de Word2Vec, y Doc2Vec, ELMo y USE hoy están superados por Sentence-BERT y sus derivados.

31.2. BERT y los embeddings contextuales

IDEA CENTRAL

En BERT (Devlin et al., 2018) el vector de una palabra depende de la oración en la que aparece.

"banco" en "Fui al banco a retirar dinero" y en "Me senté en el banco de la plaza" da dos vectores distintos. ELMo lo había mostrado con LSTM bidireccionales; BERT lo hizo con Transformers y se volvió el estándar. El código del material carga bert-base-multilingual-cased, toma el vector de la palabra en last_hidden_state (el primer subtoken, porque WordPiece puede partir palabras) y compara con coseno. La similitud entre los dos "banco" debería salir claramente menor que 1; Word2Vec daría el mismo vector.



Clave

Estático (Word2Vec, GloVe, FastText) = un vector por palabra. Contextual (ELMo, BERT) = un vector por palabra en cada oración. Solo los contextuales resuelven la polisemia.

31.3. Resumen comparativo de las representaciones

Modelo o método	Enfoque	Similitud semántica	Coseno útil	Contextual
One-hot	Palabra	No	No (ortogonales)	No
Count / Hash / TF-IDF	Documento	No	Sí	No
Word2Vec	Palabra	Sí	Sí	No
FastText	Palabra + subpalabra	Sí (+ OOV y morfología)	Sí	No
BERT	Palabra en contexto	Sí	Sí	Sí
Sentence-BERT	Oración	Sí	Sí	Sí

32. Visualización de embeddings

IDEA CENTRAL

Los embeddings tienen cientos de dimensiones (100, 300, 768 o más). Para verlos se reducen a 2 o 3 con técnicas de reducción de dimensionalidad.

Concepto	Qué hace	Cuándo se usa
t-SNE (t-Distributed Stochastic Neighbor Embedding)	Reduce dimensiones intentando conservar las relaciones de vecindad entre puntos	Ver cómo se agrupan las palabras
PCA (Principal Component Analysis, análisis de componentes principales)	Cambia a un sistema de coordenadas donde las primeras componentes capturan la mayor variación. Se toman las 2 o 3 primeras	Ver la estructura global de los vectores

El material reduce con t-SNE los vectores Word2Vec de rey, reina, hombre, mujer, niño, niña, príncipe y princesa, y con PCA los embeddings de oraciones de Sentence-BERT, donde quedan juntas las oraciones del mismo tema.

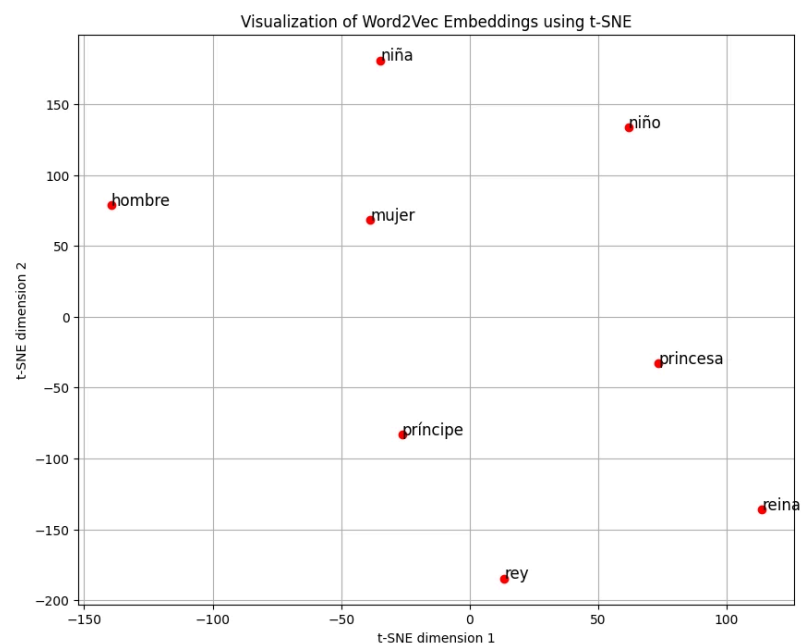


Figura 17. Vectores Word2Vec de ocho palabras reducidos a dos dimensiones con t-SNE (perplexity=6).

Fuente: material de la Unidad 2.

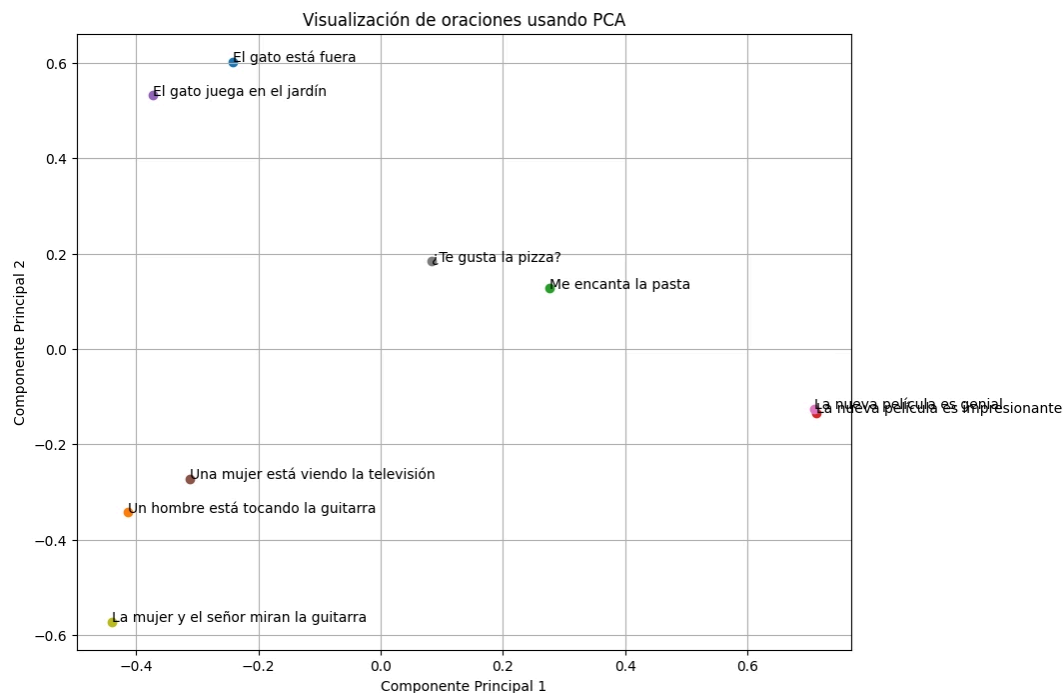


Figura 18. Embeddings de oraciones de Sentence-BERT reducidos a dos dimensiones con PCA. Las oraciones del mismo tema quedan cerca. Fuente: material de la Unidad 2.

[Explicación propia] *El ejemplo usa perplexity=6 porque en t-SNE ese valor debe ser menor que la cantidad de puntos, y acá hay 8 palabras.*

33. Embeddings de oraciones y Sentence-BERT

IDEA CENTRAL

Un embedding de oración (sentence embedding) representa una oración, un párrafo o un documento completo con un único vector de tamaño fijo que captura su significado.

CARACTERÍSTICAS SEGÚN EL MATERIAL

Se usan en clasificación, búsqueda semántica, respuesta a preguntas y traducción. Dan una representación única para oraciones de cualquier largo. La dificultad es comprimir hechos, opiniones y emociones en un vector fijo.

33.1. Promediar word embeddings: el método ingenuo

Promediar los vectores de las palabras es lo más simple. El material muestra sus fallas con spaCy (en_core_web_md):

- Pérdida de información: "It is cool" contra la palabra "It" da 0,84 de similitud, igual que contra "bad".
- El orden no afecta: "this is cool" contra "is this cool" da 1,0.

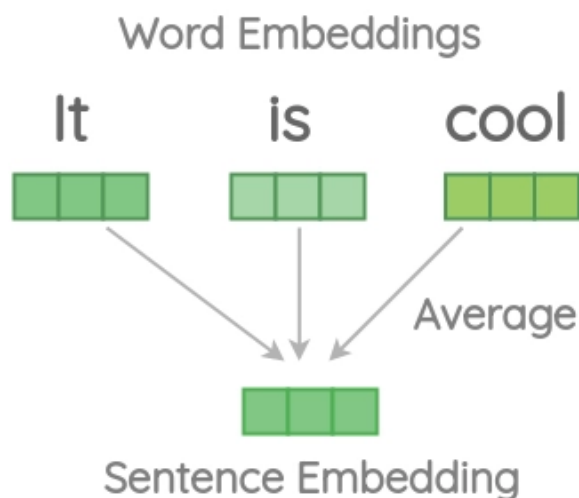


Figura 19. Embedding de oración por promedio: los vectores de "It", "is" y "cool" se promedian en un solo vector. Así se pierde el orden de las palabras. Fuente: material de la Unidad 2.

Se puede mejorar a mano (quitar stopwords, ponderar con TF-IDF, agregar n-gramas) o entrenar un modelo de punta a punta (end-to-end) que produzca el vector de la oración, como intentaron Doc2Vec, InferSent y USE.

33.2. Sentence-BERT (SBERT)

DEFINICIÓN FORMAL

Sentence-BERT (librería `sentence-transformers`) modifica BERT para producir embeddings de oración comparables con similitud de coseno. Usa redes siamesas y tripletas en el entrenamiento.

POR QUÉ HACE FALTA

Para comparar dos oraciones, BERT usa un cross-encoder: procesa cada par junto y devuelve un puntaje. Con miles de oraciones hay que procesar cada par, y se vuelve inviable. SBERT calcula un vector por oración una sola vez y compara con coseno. Según el material, el tiempo baja de horas a segundos.

EJEMPLO DEL MATERIAL

Con `SentenceTransformer('distiluse-base-multilingual-cased-v1')`, `modelo.encode(oraciones)` y `util.cos_sim()`:

Oración 1	Oración 2	Similitud
La nueva película es impresionante	La nueva película es genial	0,9809
Un hombre está tocando la guitarra	La mujer y el señor miran la guitarra	0,6964
El gato está fuera	El gato juega en el jardín	0,5979
Me encanta la pasta	¿Te gusta la pizza?	0,3107

Con palabras distintas ("impresionante", "genial"), el modelo detecta que las oraciones dicen lo mismo.

🔑 Clave

Promediar word embeddings pierde información e ignora el orden. Sentence-BERT genera un vector por oración que se compara con coseno, y eso hace práctica la búsqueda semántica sobre muchas oraciones. BERT con cross-encoder compara pares y no escala.

33.3. MTEB: cómo elegir un modelo

El MTEB (Massive Text Embedding Benchmark) evalúa modelos de embeddings con 58 conjuntos de datos, 112 idiomas y 8 tareas (entre ellas clasificación, clustering, recuperación o retrieval, reranking, similitud textual semántica y resumen). Publica un leaderboard en Hugging Face.

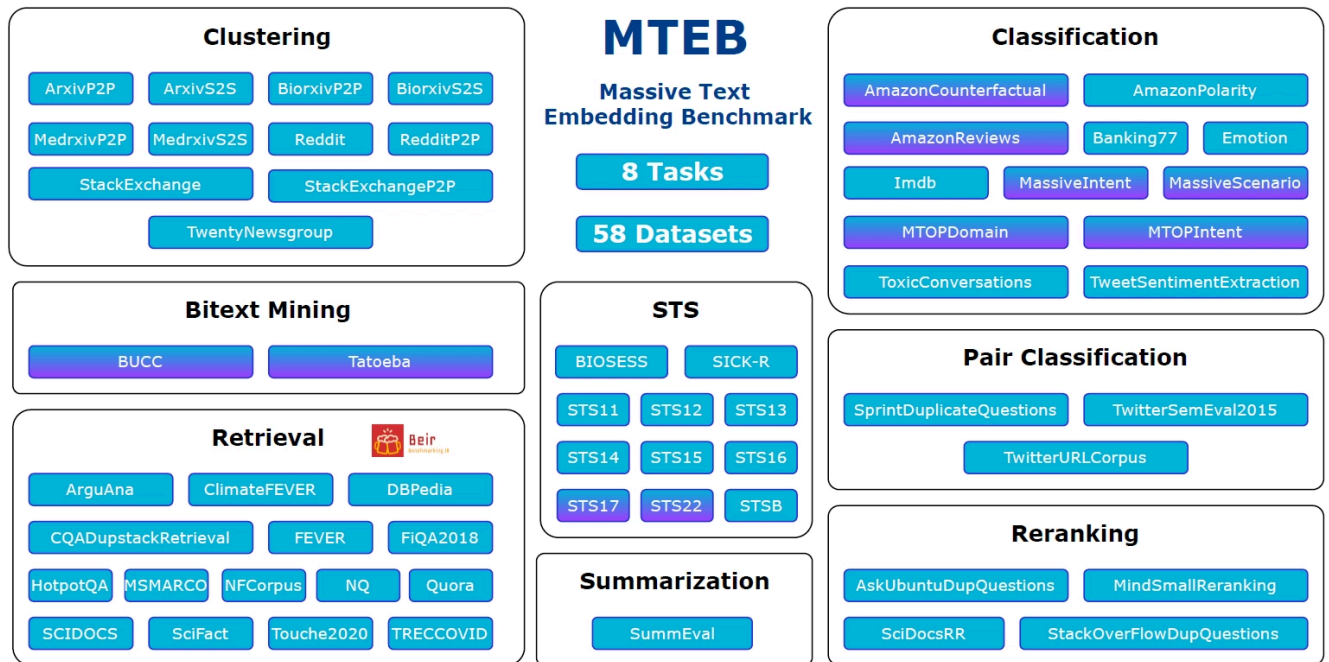


Figura 20. Tareas y conjuntos de datos del benchmark MTEB: 8 tareas y 58 conjuntos de datos. Fuente: material de la Unidad 2.

Criterio del material: no hace falta entrenar. Se elige del leaderboard un modelo multilingüe o en español, se mira su tamaño (memoria y latencia), se valida con algunos pares y se carga con `SentenceTransformer("nombre-del-modelo")`.

33.4. Cómo aprenden los modelos de embeddings modernos

Se entrenan con muchísimo texto en tareas que obligan a entender el contexto, como predecir palabras ocultas (Masked Language Modeling, modelado de lenguaje enmascarado). El entrenamiento acerca los vectores de textos con significado parecido aunque usen otras palabras u otro idioma: "coche" y "automóvil" quedan cerca. El modelo no entiende como una persona; aprende patrones estadísticos que alcanzan para agrupar textos relacionados.

Parte IV. Herramientas de estudio

Glosario

Término	Definición breve
Bigrama / trigrama	N-grama de 2 / 3 palabras.
CBOW	Algoritmo de Word2Vec que predice la palabra central a partir de su contexto.
Chunk (fragmento)	Pedazo de texto resultante de la segmentación.
chunk_overlap	Caracteres del final de un fragmento que se repiten al inicio del siguiente.
chunk_size	Tamaño máximo de cada fragmento.
Colisión de hash	Dos entradas distintas que producen el mismo valor de hash.
Co-ocurrencia	Aparición conjunta de dos palabras dentro de una ventana o unidad de texto.
Corpus	Conjunto de documentos que se analiza.
Correlación de palabras	Medida estadística de si la presencia de un término se asocia con la de otro, considerando también la ausencia.
Count Vectorizer	Representa cada documento con el conteo de cada palabra del vocabulario.
Cross-encoder	Modelo que procesa dos oraciones juntas para dar un puntaje de similitud. No escala a muchas oraciones.
Embedding	Vector denso aprendido que representa el significado de una palabra u oración.
Embedding contextual	Embedding que cambia según la oración (ELMo, BERT).
Encoding (codificación)	Tabla que asigna bytes a caracteres (UTF-8, ISO-8859-1).
Extracción de información	Obtener entidades, relaciones o eventos de un texto ya extraído.
Extracción de texto	Recuperar texto plano de un contenedor (PDF, HTML, audio, escaneo).
FastText	Extensión de Word2Vec que usa n-gramas de caracteres. Maneja morfología y OOV.
Función de hash	Función que transforma una entrada en un valor de longitud fija.
IDF	Frecuencia inversa de documento: log del total de documentos sobre los que contienen el término.
Lematización	Reducir una palabra a su lema válido usando diccionario y análisis morfológico.
Matriz dispersa	Matriz donde la mayoría de los valores son cero.
MTEB	Benchmark que compara modelos de embeddings en muchas tareas e idiomas.
N-grama	Secuencia contigua de n palabras (o de n caracteres, en FastText).
Normalización NFKD	Descomposición Unicode de un carácter en su base más sus marcas combinantes.
OCR	Reconocimiento óptico de caracteres: obtener texto de una imagen.
One-hot encoding	Vector con un 1 en la posición de la palabra y 0 en el resto.
OOV	Palabra fuera de vocabulario, que el modelo no vio al entrenar.
Parseo	Analizar un texto con reglas o patrones para extraer datos estructurados.
PCA	Reducción de dimensionalidad que conserva las direcciones de mayor variación.
Rasgo semántico	Dimensión que representa parte del significado de una palabra (género, edad, realeza).
Segmentación (chunking)	Dividir un texto en fragmentos más chicos.
Segmentación semántica	Segmentación que corta donde cambia el tema, medido con embeddings.
Sentence-BERT	Modificación de BERT que produce un embedding por oración, comparable con coseno.

Término	Definición breve
Similitud de coseno	Coseno del ángulo entre dos vectores: producto escalar sobre producto de normas.
Skip-gram	Algoritmo de Word2Vec que predice el contexto a partir de la palabra central.
Stemming	Recortar sufijos con reglas para llegar a una raíz, que puede no ser una palabra real.
Stopwords	Palabras muy frecuentes que en ciertos contextos aportan poco ("el", "de").
t-SNE	Reducción de dimensionalidad que conserva las vecindades entre puntos.
TF	Frecuencia de término: apariciones de la palabra sobre total de palabras del documento.
TF-IDF	Producto de TF e IDF. Pesa alto lo frecuente en el documento y raro en el corpus.
Token	Unidad mínima en que se divide un texto (oración, palabra o subpalabra).
Vocabulario	Conjunto de palabras distintas del corpus.
Web scraping	Extraer información de páginas web leyendo su HTML.
Word2Vec	Modelo de 2013 que aprende embeddings estáticos con Skip-gram o CBOW.

Página de repaso

1. Extracción de texto $\neq$ extracción de información. La primera saca el texto del contenedor; la segunda saca datos estructurados del texto.
2. Conocer el encoding antes de leer. UTF-8 usa más bytes para ¿, ó, á: "¿Cómo estás?" = 15 bytes en UTF-8 y 12 en ISO-8859-1.
3. BeautifulSoup .text borra todas las etiquetas, incluso las que marcan párrafos.
4. PDF guarda instrucciones de dibujo. Escaneado $\rightarrow$ pdf2image $\rightarrow$ pytesseract.
5. EasyOCR devuelve texto y ubicación; pytesseract solo texto.
6. Conversores: Markdown para entrar, Docling para estructura local, MinerU para papers y escaneos, Marker con GPU.
7. Audio: SpeechRecognition, Whisper (multitarea) o subtítulos de YouTube. Video $\rightarrow$ extraer audio con ffmpeg.
8. Scraping: respetar Términos de Servicio y robots.txt. Session conserva cookies; Playwright/Selenium para páginas con JavaScript.
9. Parseo: split (frágil), regex (flexible), parse (formatos fijos).
10. Normalizar: minúsculas, puntuación con `[^\w\s]`, acentos con NFKD + filtrar combinantes.
11. Stopwords: reducen ruido y dimensión, pero negaciones e intensificadores cambian el sentido.
12. Abreviaturas antes que corrección ortográfica. TextBlob no sirve para español.
13. Tokenizar es elegir la unidad de análisis. Tokenizar no borra: segmenta.
14. Stemming: rápido, raíces inválidas ("perr"). Lematización: lenta, lemas válidos ("perro").
15. Emojis: eliminar, convertir a texto o tratar como palabras.
16. N-gramas predicen la palabra siguiente, pero solo con secuencias vistas.
17. Co-ocurrencia cuenta; correlación mide tendencia estadística con presencia y ausencia.
18. Chunks: todos $\leq$ chunk_size. Overlap preserva contexto. Recursiva prueba "\n\n", "\n", " ", "".
19. One-hot: vectores ortogonales, sin similitud.
20. Count Vectorizer: bolsa de palabras, ignora el orden.
21. TF-IDF = TF $\times$ IDF. Palabra en todos los documentos $\rightarrow$ IDF = 0.
22. Hashing: sin vocabulario, tamaño fijo, colisiones, no reversible.
23. Embeddings: densos, aprendidos, cercanía = significado parecido.
24. Coseno: 1 igual dirección, 0 ortogonal, -1 opuesto. Ignora la magnitud.
25. Skip-gram: palabra $\rightarrow$ contexto. CBOW: contexto $\rightarrow$ palabra. gensim sg=1 / sg=0.
26. rey - hombre + mujer $\approx$ reina.
27. FastText: n-gramas de caracteres, resuelve OOV y morfología; sigue siendo estático.
28. BERT: contextual, "banco" cambia según la oración.
29. Promediar embeddings pierde información e ignora el orden. SBERT: un vector por oración, comparable con coseno.
30. Elegir modelo con MTEB: multilingüe o español, tamaño, validar con ejemplos.

Preguntas de autoevaluación

1. ¿Qué diferencia hay entre extracción de texto y extracción de información? Da un ejemplo de cada una.
2. ¿Por qué la cadena "¿Cómo estás?" ocupa 15 bytes en UTF-8 y 12 en ISO-8859-1? ¿Qué pasa si se lee con el encoding equivocado?
3. Tenés un PDF escaneado de 40 páginas. Describí el proceso para obtener su texto y nombrá las herramientas de cada paso.
4. ¿En qué casos eliminar stopwords puede empeorar un análisis de sentimiento? Da dos ejemplos de palabras que no conviene quitar.
5. Compará stemming y lematización con la palabra "corriendo". ¿Cuándo elegirías cada uno?
6. Explicá qué hacen `chunk_size` y `chunk_overlap` en `RecursiveCharacterTextSplitter`. ¿Por qué un fragmento puede quedar más corto que `chunk_size`?
7. En un corpus de 4 documentos, la palabra "red" aparece 3 veces en un documento de 12 palabras y está en 2 de los 4 documentos. Calculá su TF, IDF (logaritmo natural, fórmula básica) y TF-IDF.
8. ¿Por qué la similitud de coseno entre dos palabras distintas codificadas con one-hot es siempre 0? ¿Qué cambia con los embeddings?
9. ¿Qué diferencia hay entre Skip-gram y CBOW? ¿Qué agrega FastText a Word2Vec y qué problema sigue sin resolver?
10. ¿Por qué promediar los word embeddings de una oración es una mala representación? ¿Cómo lo resuelve Sentence-BERT y por qué es más eficiente que comparar pares con BERT?

Observaciones sobre el material

Estos puntos del material parecen ambiguos o incorrectos. En el cuerpo del resumen se mantuvo lo que dice el material; acá se explica la duda.

1. **Analogía rey/reina (Unidad 2).** El texto escribe "hombre – mujer + rey = reina". La operación correcta es $\text{rey} - \text{hombre} + \text{mujer} \approx \text{reina}$, que es lo que hace el código del propio material (`positive=['mujer', 'rey'], negative=['hombre']`).
2. **Gráfico "Cosine in LLM" (Unidad 2).** Rotula $64,65^\circ$ y $0,4284$ como el ángulo entre Apple y Dog. Con los vectores del gráfico, ese valor corresponde a Apple $[10, 50]$ contra Cat $[48, 12]$. Apple contra Dog $[50, 10]$ da $67,4^\circ$ y coseno $0,385$. La conclusión (no son similares) no cambia.
3. **Sentence-BERT (Unidad 2).** El texto dice que fue "desarrollada originalmente por Google en 2020". La línea temporal del mismo material la ubica en 2019. Hay una inconsistencia interna.
4. **Fórmula de IDF (Unidad 2).** La fórmula presentada es $\log(N / df)$ sin indicar la base. La cuenta posterior usa la versión suavizada de scikit-learn, $\ln((1+N)/(1+df)) + 1$, sin aclarar que es otra fórmula. Los valores de la matriz TF-IDF además están normalizados (norma L2), y el material no lo dice.
5. **Correlación (Unidad 1).** Dice que una correlación alta indica que el otro término tiende a aparecer (positiva) "o a no aparecer (correlación nula)". Que un término tienda a no aparecer cuando el otro aparece es correlación negativa; nula significa sin relación. La lista de valores de Pearson tampoco menciona el -1.
6. **ISO-8859-1 y Latin-1 (Unidad 1).** El texto las presenta como codificaciones distintas ("si estuviera en Latin-1, usaríamos encoding='latin-1'"). Son dos nombres de la misma codificación.

7. **Ejemplo bueno/mejor (Unidad 1).** Dice que el stemming lleva "bueno, buenísimo, mejor" a "buen", y que la lematización lleva "mejor" a "bueno". Un stemmer recorta sufijos, así que no puede convertir "mejor" en "buen". Que un lematizador lleve "mejor" a "bueno" depende del modelo; el material no muestra la salida. Tomalo como una ilustración de la pérdida de matices.
8. **Título de sección (Unidad 1).** "Texto desde imágenes o video" trata en realidad sobre audio y video.
9. **Stopwords (Unidad 1).** "Si eliminamos las palabras menos comunes" parece un error por "más comunes".
10. **Ejemplo de BERT (Unidad 2).** El material no muestra el valor numérico de la similitud banco-dinero contra banco-plaza; solo dice lo que "debería" salir.